

Parallel Programming Beyond the CPU

Tim Mattson



Tim demonstrates his new invention: Kayak snorkeling. Palawan Philippines, 2019

An brief aside about the origins of parallel programming models

The Bad Old days ...

Parallel programming environments in the 90's

ABCPL	CORRELATE	GLU	Mentat	Parafraise2	pC++
ACE	CPS	GUARD	Legion	Paralation	SCHEDULE
ACT++	CRL	HASL	Meta Chaos	Parallel-C++	SciTL
Active messages	CSP	Haskell	Midway	Parallaxis	POET
Adl	Cthreads	HPC++	Millipede	ParC	SDDA.
Adsmith	CUMULVS	JAVAR.	CparPar	ParLib++	SHMEM
ADDAP	DAGGER	MIRUS	Mirage	ParLin	SIMPLE
AFAPI	DAPPLE	HPC	MpC	Parmacs	Sina
ALWAN	Data Parallel C	HPF	MOSIX	Parti	SISAL.
AM	DC++	IMPACT	Modula-P	pC	distributed smalltalk
AMDC	DCE++	ISIS.	Modula-2*	pC++	SMI.
AppLeS	DDD	JAVAR	Multipol	PCN	SONiC
Amoeba	DICE.	JADE	MPI	PCP:	Split-C.
ARTS	DIPC	Java RMI	MPC++	PH	SR
Athapascan-0b	DOLIB	javaPG	Munin	PEACE	Sthreads
Aurora	DOIME	JavaSpace	Nano-Threads	PCU	Strand.
Automap	DOSMOS.	JIDL	NESL	PET	SUIF.
bb_threads	DRL	Joyce	NetClasses++	PETSc	Synergy
Blaze	DSM-Threads	Khoros	Nexus	PENNY	Telegrphos
BSP	Ease .	Karma	Nimrod	Phosphorus	SuperPascal
BlockComm	ECO	KOAN/Fortran-S	NOW	POET.	TCGMSG.
C*.	Eiffel	LAM	Objective Linda	Polaris	Threads.h++.
"C* in C	Eilean	Lilac	Occam	POOMA	TreadMarks
C**	Emerald	Linda	Omega	POOL-T	TRAPPER
CarlOS	EPL	JADA	OpenMP	PRESTO	uC++
Cashmere	Excalibur	WWWinda	Orca	P-RIO	UNITY
C4	Express	ISETL-Linda	OOF90	Prospero	UC
CC++	Falcon	ParLin	P++	Proteus	V
Chu	Filaments	Eilean	P3L	QPC++	ViC*
Charlotte	FM	P4-Linda	p4-Linda	PVM	Visifold V-NUS
Charm	FLASH	Glenda	Pablo	PSI	VPE
Charm++	The FORCE	POSYBL	PADE	PSDM	Win32 threads
Cid	Fork	Objective-Linda	PADRE	Quake	WinPar
Cilk	Fortran-M	LiPS	Panda	Quark	WWWinda
CM-Fortran	FX	Locust	Papers	Quick Threads	XENOOPS
Converse	GA	Lparx	AFAPL	Sage++	XPC
Code	GAMMA	Lucid	Para++	SCANDAL	Zounds
COOL	Glenda	Maisie	Paradigm	SAM	ZPL
		Manifold			

A combination of vendors wanting to lock-down applications and the fact that its "fun" to create new programming environments created chaos for application developers

History of MPI

Workstation vendors wanted into the HPC market

PVM was great but didn't support quality SW engineering

Hardware:

By the early 90's, massively parallel processors (MPPs) and the new trend with clusters convinced even the skeptics that the "killer micros" had won.

MPP Vendors

Needed a common foundation to build a parallel SW industry

User Community

Fed-up recoding as they moved between platforms

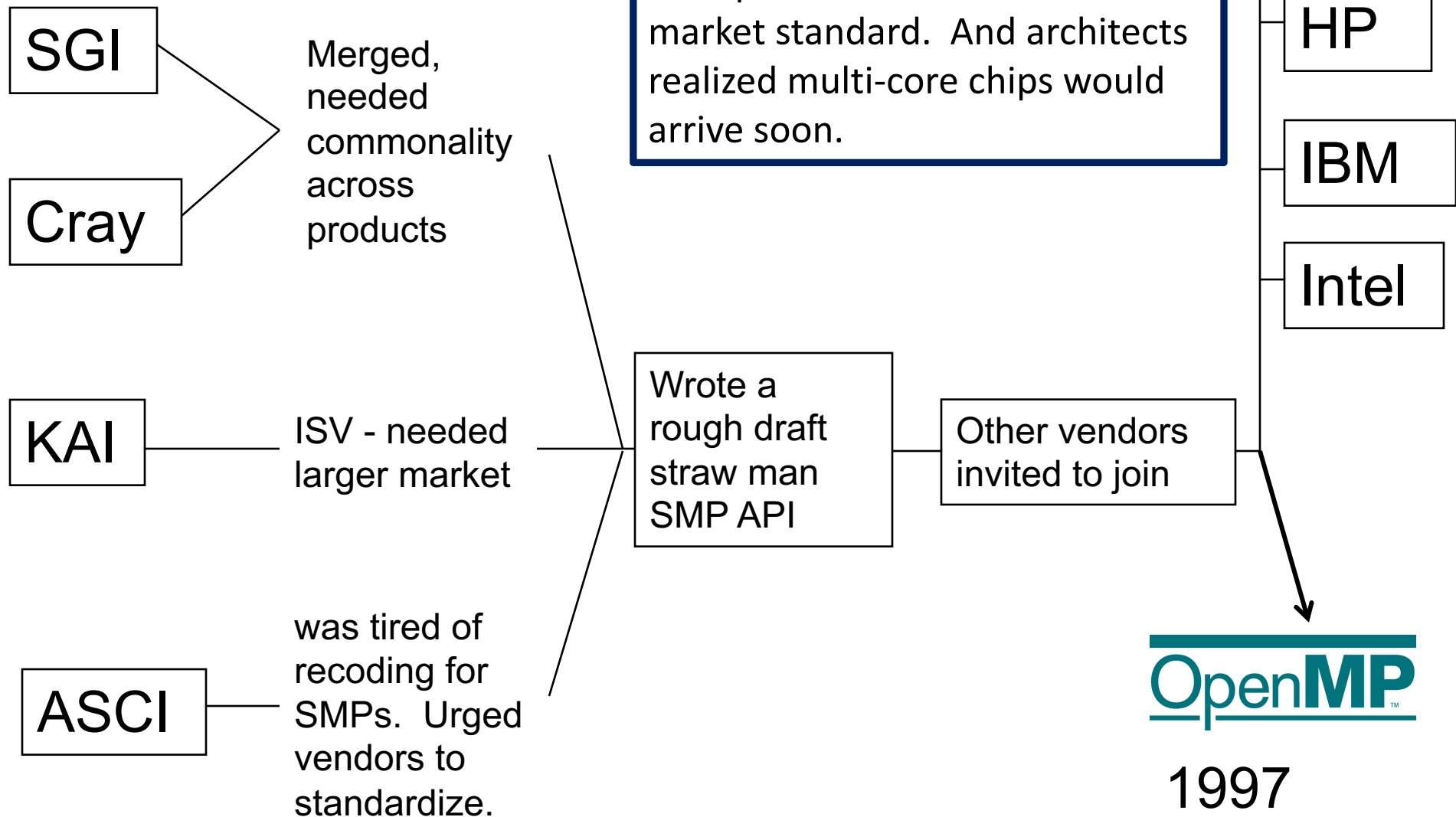
After several years of informal discussions, the MPI forum was created in 1992. A draft specification was presented one year later at SC'93.



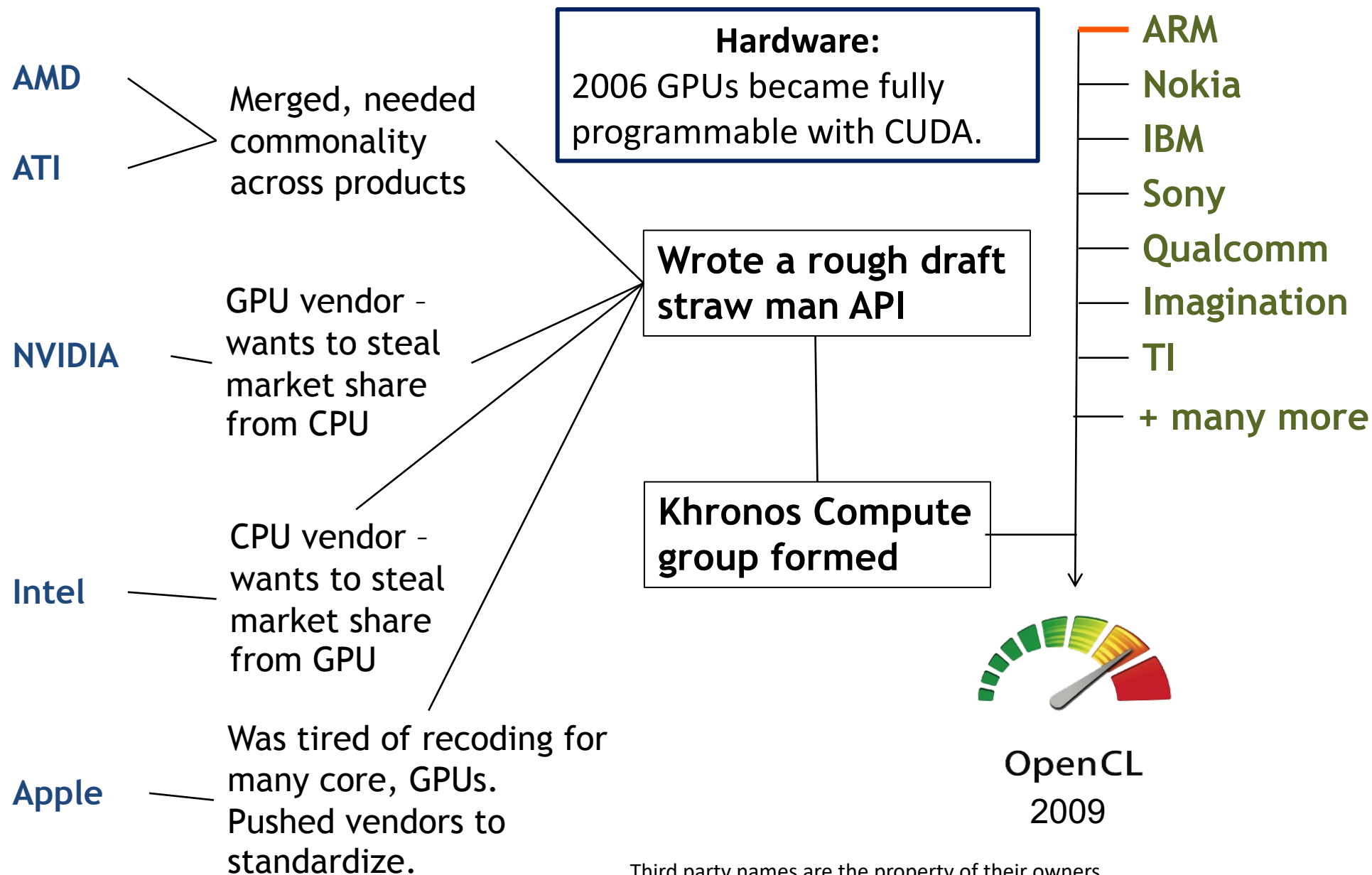
1994

Many of us worked in the MPI forum ... leadership came from the DOE National Labs. In particular, the reference implementation from Bill Gropp and Rusty Lusk of Argonne national lab called MPIch helped us get it right in the 1.0 specification and made sure a working implementation of the standard was available right from the beginning.

History of OpenMP



The origins of OpenCL

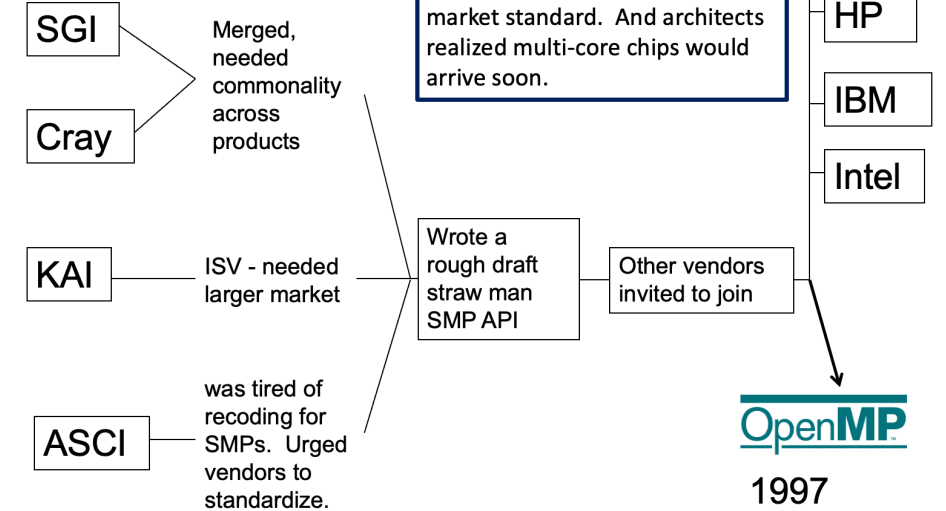


Third party names are the property of their owners.

Key Lesson for you

- Application developers in HPC are the community that really matters You have MUCH MORE power than you think, but only:
 - You are willing to speak with One Voice.
 - You refuse to be trapped in a walled Garden.
 - Vendors will try to divide and conquer you. Hold firm or all is lost!

History of OpenMP



History of MPI

Workstation vendors wanted into the HPC market

PVM was great but didn't support quality SW engineering

Hardware:

By the early 90's, massively parallel processors (MPPs) and the new trend with clusters convinced even the skeptics that the "killer micros" had won.

MPP Vendors

Needed a common foundation to build a parallel SW industry

User Community

Fed-up recoding as they moved between platforms

After several years of informal discussions, the MPI forum was created in 1992. A draft specification was presented one year later at SC'93.



1994

Many of us worked in the MPI forum ... leadership came from the DOE National Labs. In particular, the reference implementation from Bill Gropp and Rusty Lusk of Argonne national lab called MPICH helped us get it right in the 1.0 specification and made sure a working implementation of the standard was available right from the beginning.

The Bad Old days ...

Parallel programming environments in the 90's

ABCPL	CORRELATE	GLU	Mentat	Parafuse2	pC++
ACE	CPS	GUARD	Legion	Paralation	SCHEDULE
ACT++	CRL	HASL	Meta Chaos	Parallel-C++	SciTL
Active messages	CSP	Haskell	Midway	Parallaxis	ScTL
Adl	Cthreads	HPC++	Millipede	ParC	SHMEM
Adsmith	CUMULVS	JAVAR	CparPar	ParLib++	SIMPLE
ADDAP	DAGGER	FORUS	Mirage	ParLin	Sina
AFAP	DAPPLE	HPC	MpC	Parmas	SISAL
ALWAN	Data Parallel C	HPF	MOSIX	Parti	distributed smalltalk
AM	DC++	IMPACT	Modula-P	pC	SML
AMDC	DCE++	ISIS	Modula-2*	pC++	SONIC
AppLeS	DDD	JAVAR	Multipol	PCN	Split-C
Amoeba	DICE	JADE	MPI	PCP	SR
ARTS	DIPC	Java RMI	MPC++	PH	Threads
Athapascan-0b	DOLIB	javaPG	Munin	PEACE	Strand.
Aurora	DOME	JavaSpace	Nano-Threads	PCU	SUIF
Automap	DOSMOS	JIDL	JOYCE	PET	Synergy
bb_threads	DRL	Joyce	Khoros	PETSc	SuperPascal
Blaze	DSM-Threads	Khoros		PENNY	TCGMSG
BSP					Threads.h++
BlockComm					TreadMarks
C*					TRAPPER
"C*" in C					
C**					
CarlOS					
Cashmere					
C4					
CC++					
Chu					
Charlotte					
Charm					
Charm++					
Cid					
Cilk					
CM-Fortran					
Converse					
Code					
COOL					

Please don't let this happen again

**EVERYTIME YOU USE CUDA OR
OPENACC**



**GOD KILLS A
KITTEN**

**... Back to our previously scheduled
program**

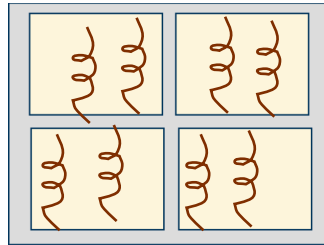
Parallel Programming Beyond the CPU

Tim Mattson

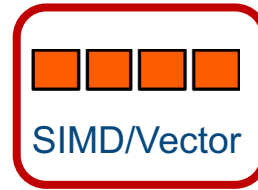


Tim demonstrates his new invention: Kayak snorkeling. Palawan Philippines, 2019

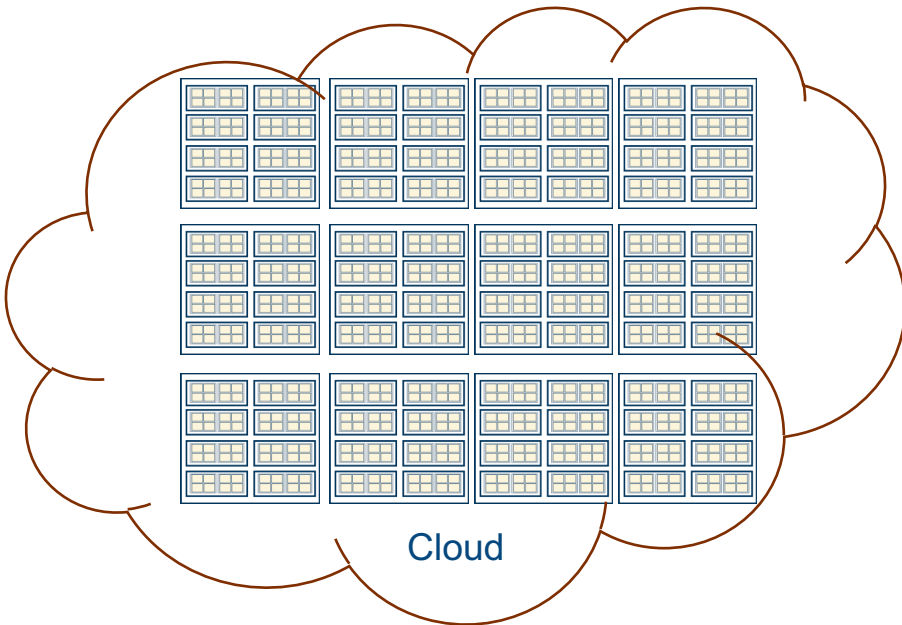
Hardware is diverse



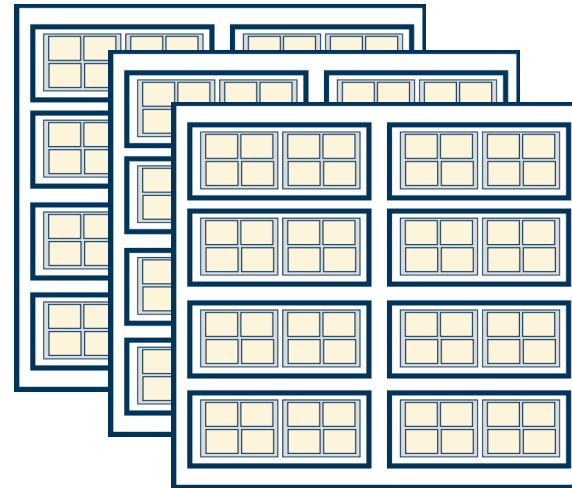
CPU



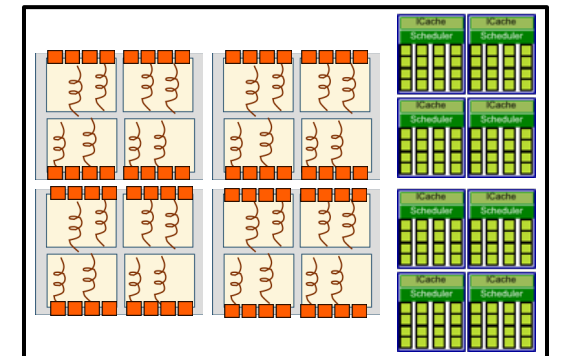
GPU



Cloud



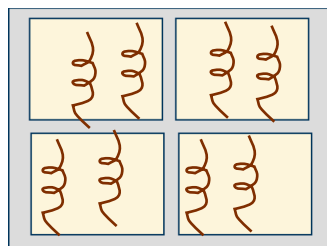
Cluster



Heterogeneous node

Hardware is diverse

You know all about programming with threads and a shared memories



CPU

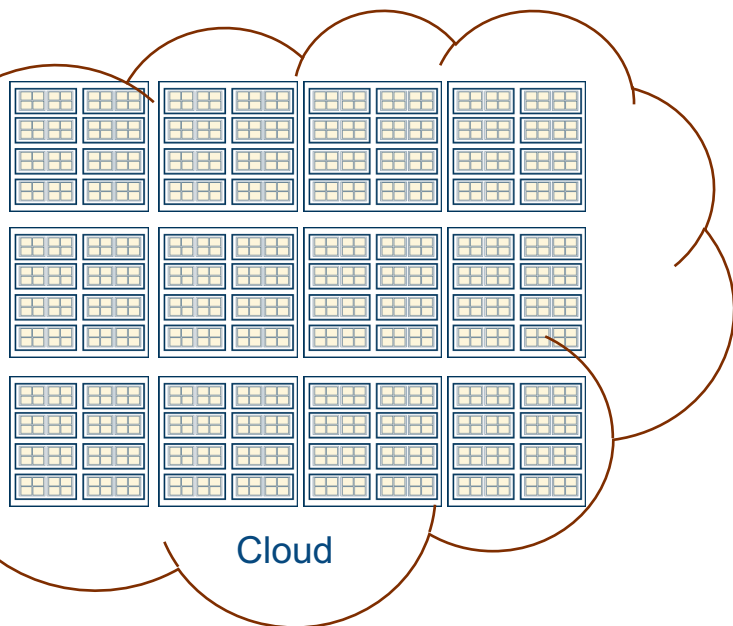
We'll leave vectorization to the compiler (use `-O3`)



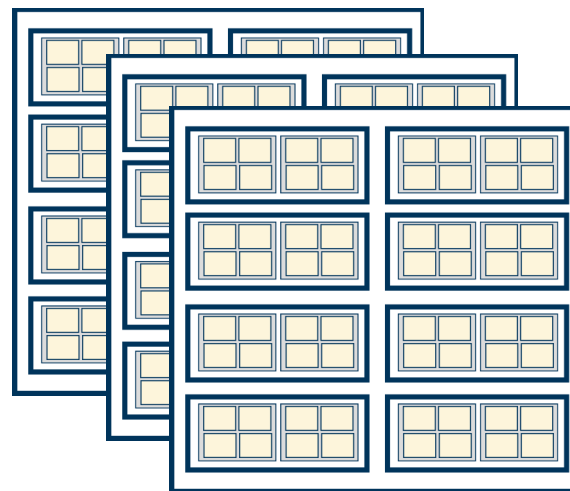
SIMD/Vector



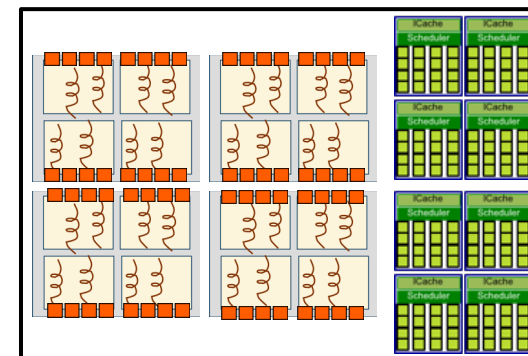
GPU



Cloud

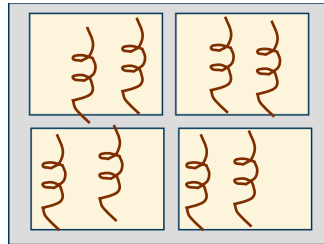


Cluster



Heterogeneous node

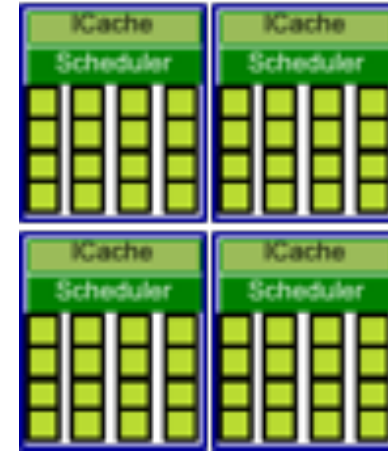
Hardware is diverse



CPU

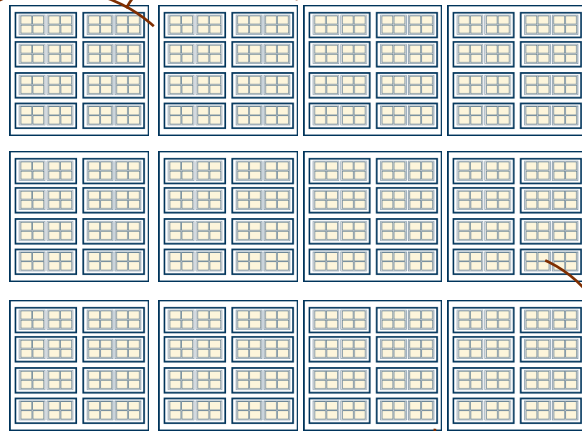


SIMD/Vector

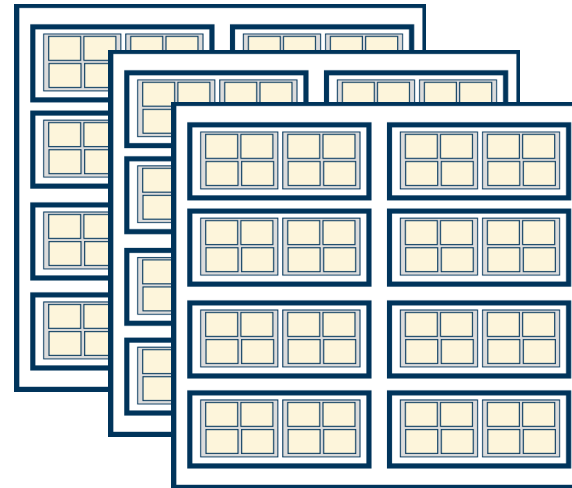


GPU

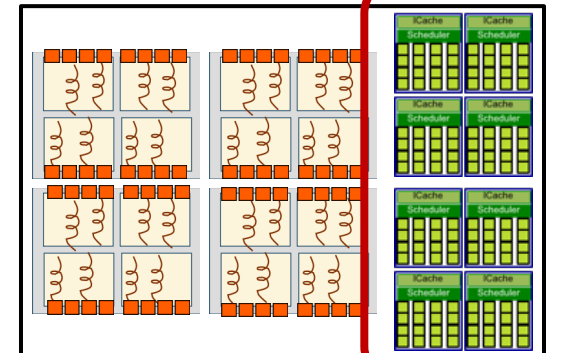
A very brief
introduction to
GPU
programming



Cloud

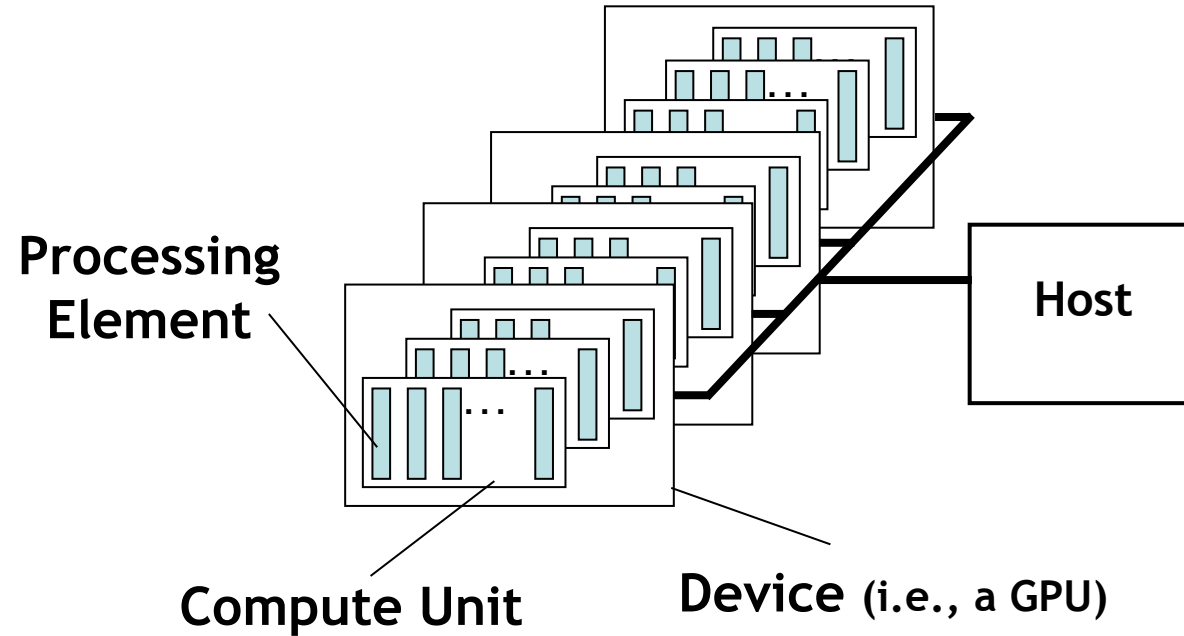


Cluster



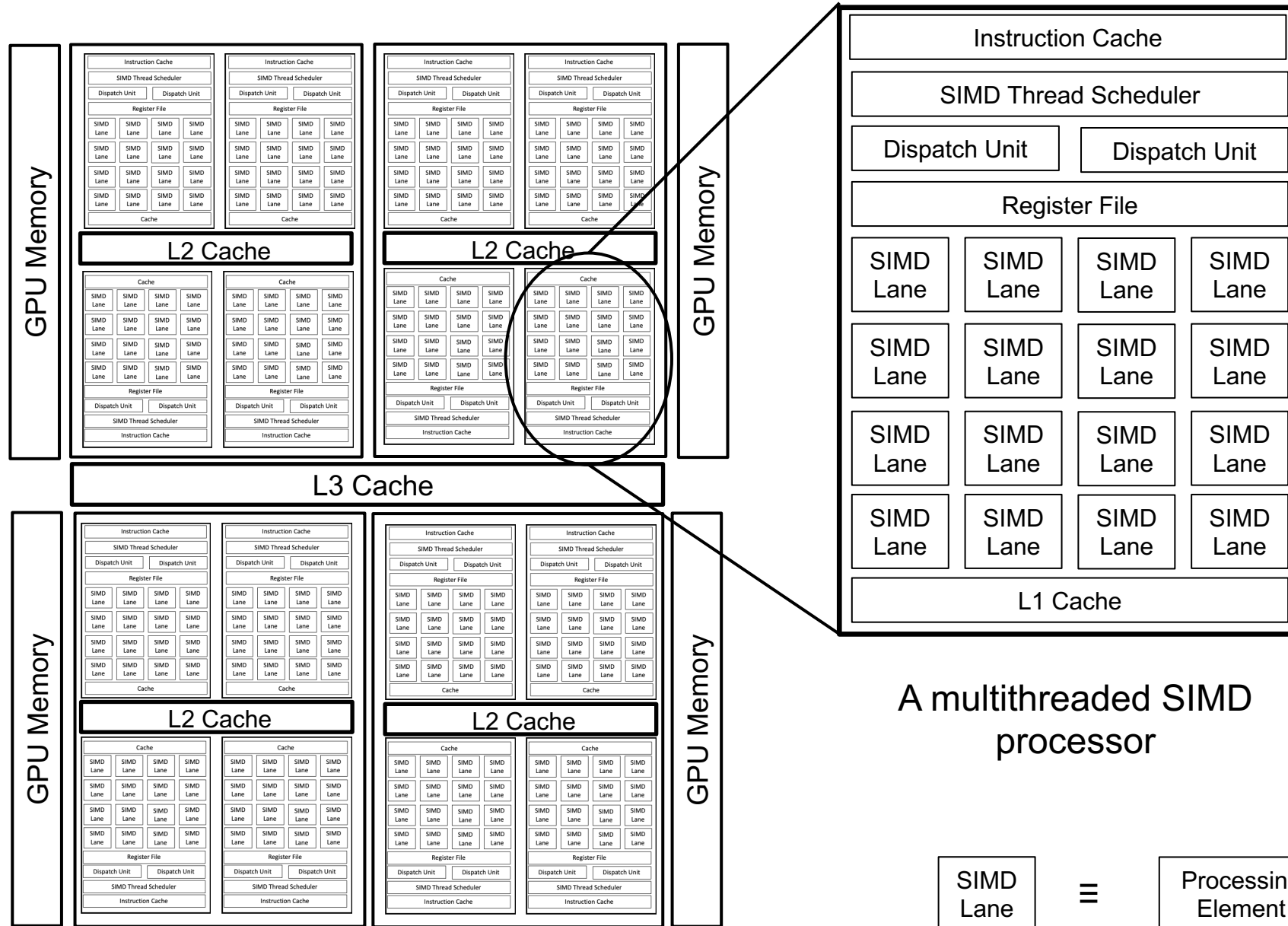
Heterogeneous node

GPU Programming: A Host/Device Platform Model



- One **Host** and one or more **Devices**
 - Each Device is composed of one or more Compute Units
 - Each Compute Unit is divided into one or more **Processing Elements**
- Memory divided into **host memory** and **device memory**

A Generic GPU (following Hennessey and Patterson)



The “BIG idea” Behind GPU programming

Data Parallel vadd with CUDA

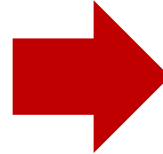
Traditional Loop based vector addition (vadd)

```
int main() {
    int N = . . . ;
    float *a, *b, *c;

    a* =(float *) malloc(N * sizeof(float));

    // ... allocate other arrays (b and c)
    // and fill with data

    for (int i=0;i<N; i++)
        c[i] = a[i] + b[i];
}
```



```
// Compute sum of length-N vectors: C = A + B
void __global__
vecAdd (float* a, float* b, float* c, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) c[i] = a[i] + b[i];
}

int main () {
    int N = ... ;
    float *a, *b, *c;
    cudaMalloc (&a, sizeof(float) * N);
    // ... allocate other arrays (b and c)
    // and fill with data

    // Use thread blocks with 256 threads each
    vecAdd <<< (N+255)/256, 256 >>> (a, b, c, N);
}
```

Assume a GPU with
unified shared memory
... allocate on host,
visible on device too

How do we execute code on a GPU: The SIMT model (Single Instruction Multiple Thread)

1. Write kernel code for the scalar work-items

```
// Compute sum of order-N matrices: C = A + B
void __global__
matAdd (float* a, float* b, float* c, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;
    if (i < N && j < N) c[i][j] = a[i][j] + b[i][j];
}

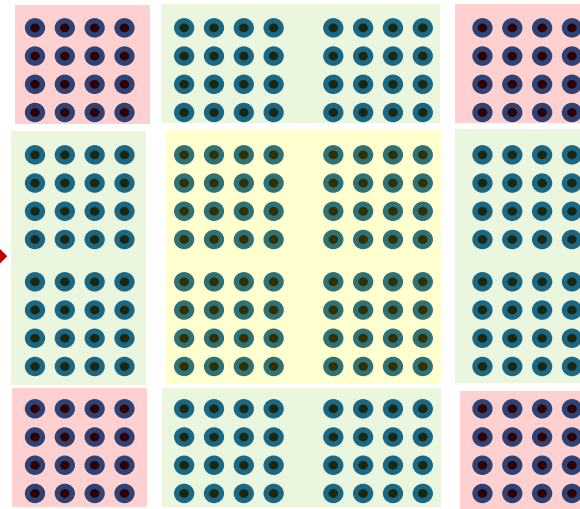
int main () {
    int N = ... ;
    float *a, *b, *c;
    cudaMalloc (&a, sizeof(float) * N);
    // ... allocate other arrays (b and c)
    // and fill with data

    // define threadBlocks and the Grid
    dim3 dimBlock(4,4);
    dim3 dimGrid(4,4);

    // Launch kernel on Grid
    matAdd <<< dimGrid,dimBlock>>> (a, b, c, N);
}
```

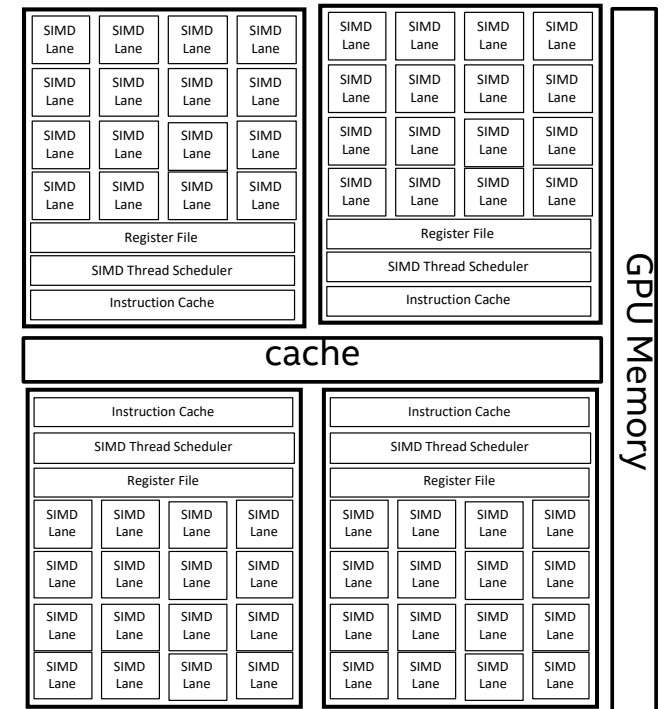
This is CUDA code

2. Map work-items onto an N dim index space.

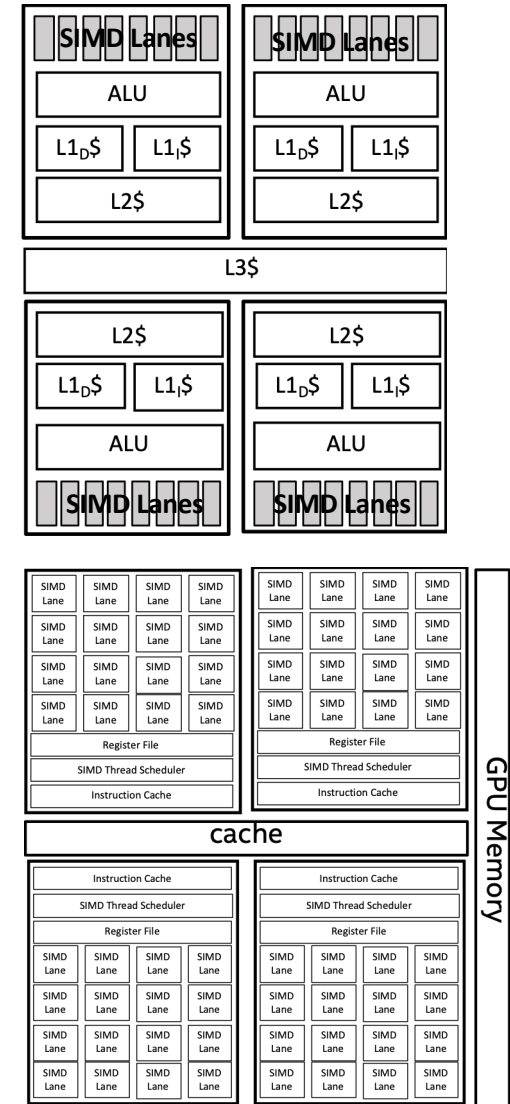
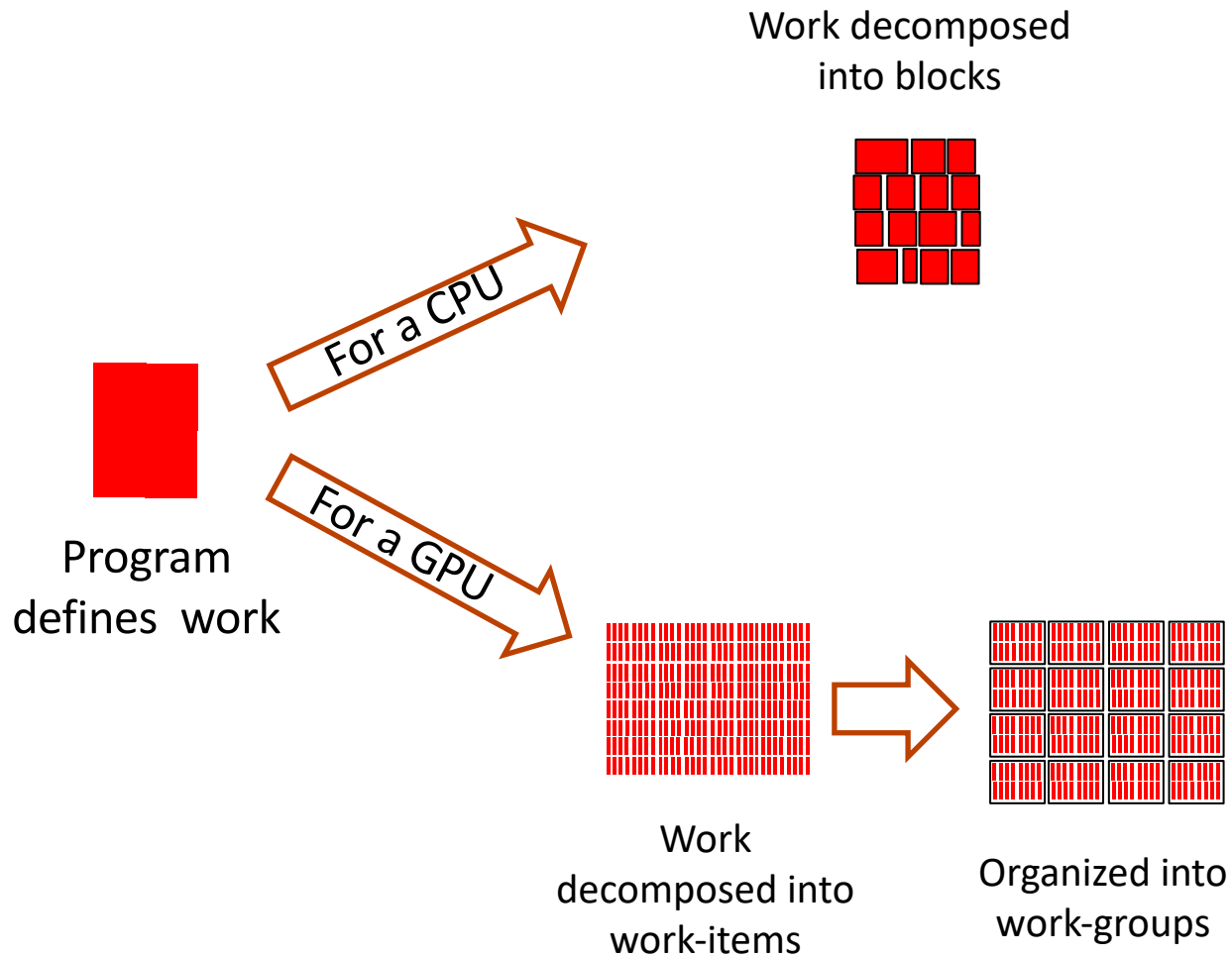


3. Map data structures onto the same index space

4. Run on hardware designed around the same SIMT execution model

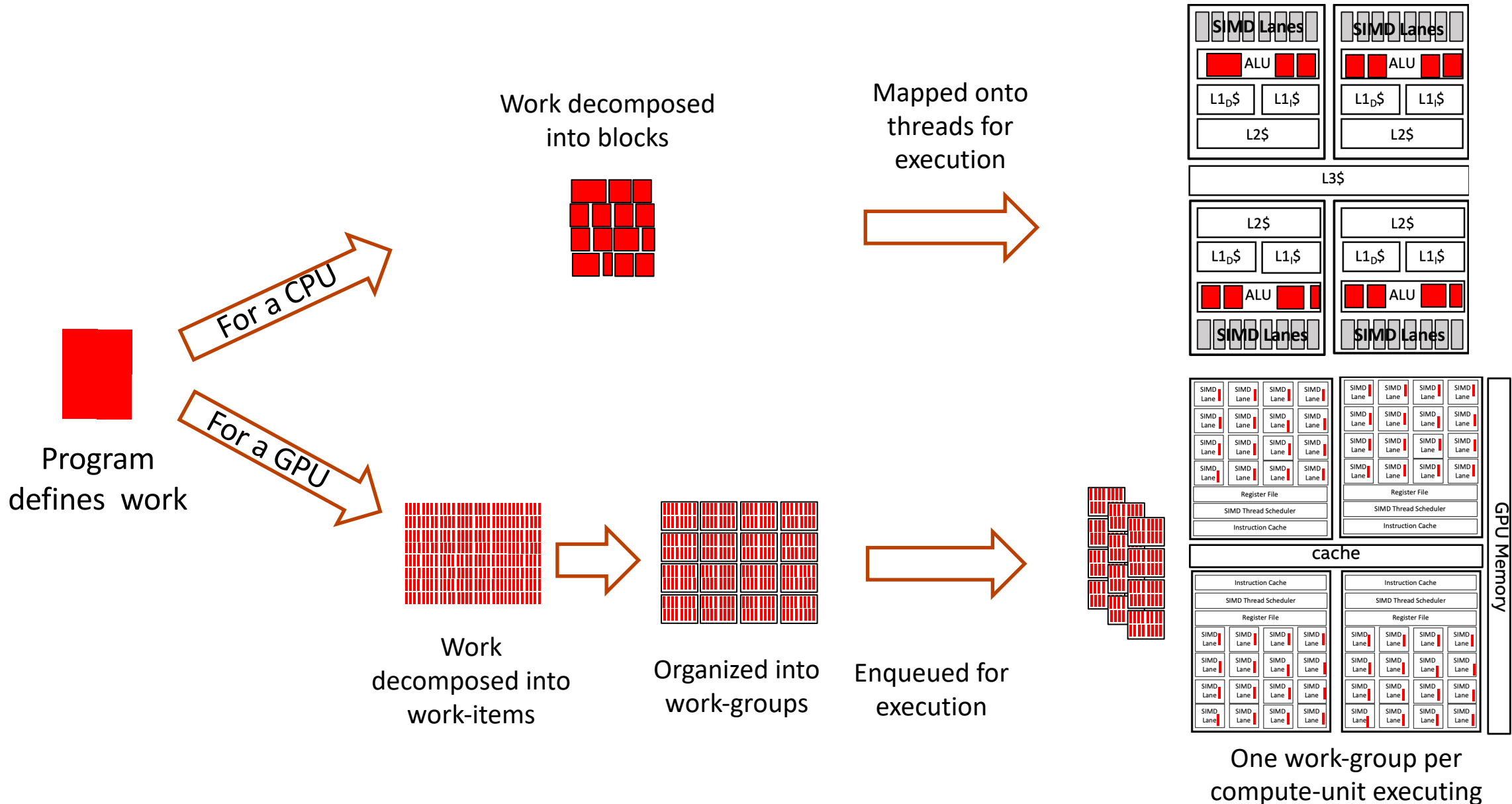


Executing a program on CPUs and GPUs



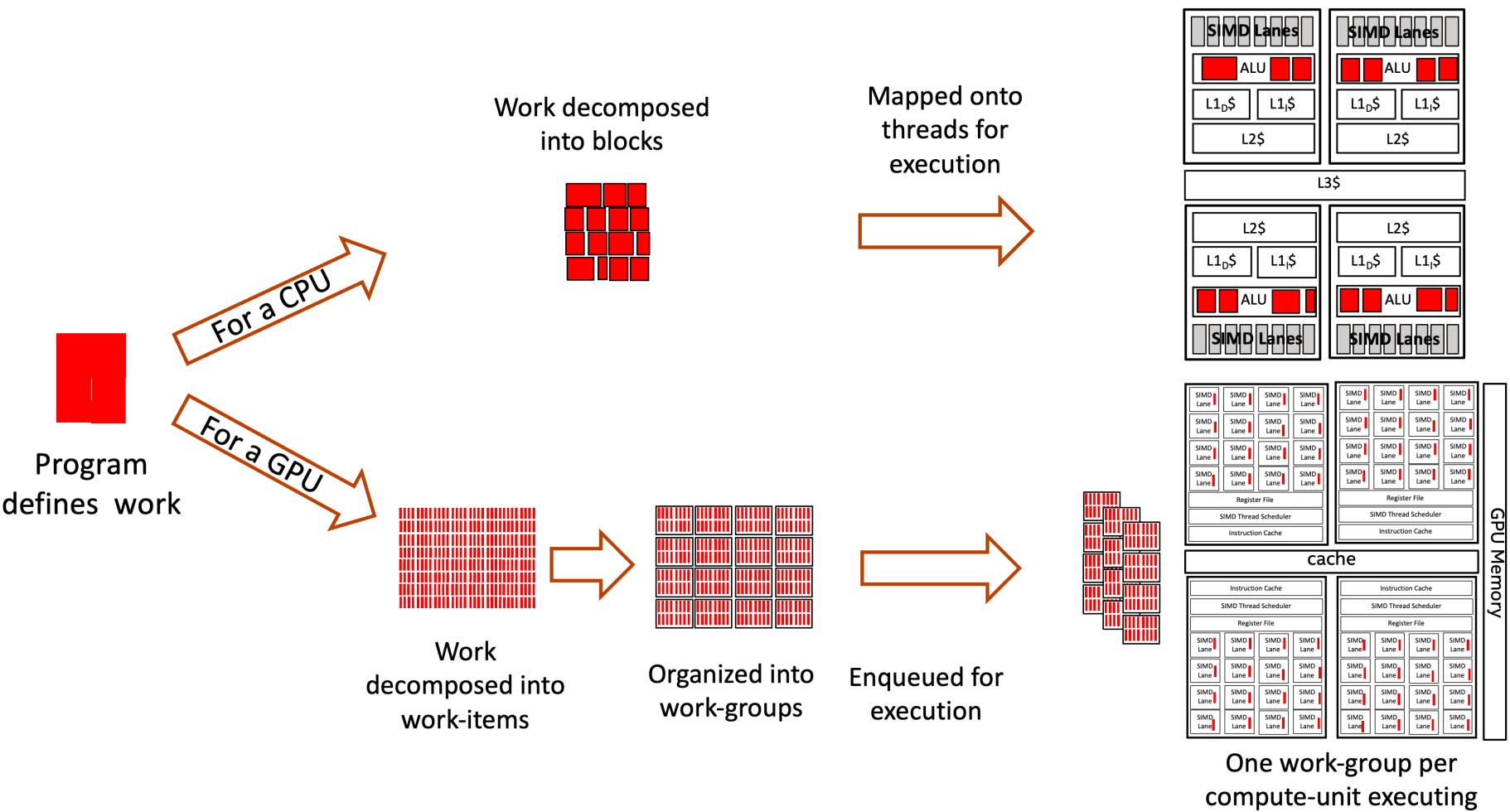
One work-group per compute-unit executing

Executing a program on CPUs and GPUs



CPU/GPU execution models

Executing a program on CPUs and GPUs



For a CPU, the threads are all active and able to make forward progress. Optimized for latency sensitive problems

For a GPU, any given work-group might be in the queue waiting to execute. Optimized for high throughput workflows

5-point stencil: the heat program

- The heat equation models changes in temperature over time.

$$\frac{\partial u}{\partial t} - \alpha \nabla^2 u = 0$$

- We'll solve this numerically on a computer using an explicit **finite difference** discretisation.
- $u = u(t, x, y)$ is a function of space and time.
- Partial differentials are approximated using diamond difference formulae:

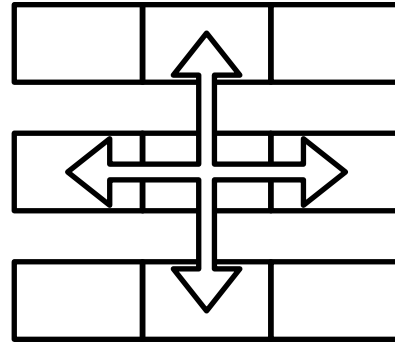
$$\frac{\partial u}{\partial t} \approx \frac{u(t + 1, x, y) - u(t, x, y)}{dt}$$

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u(t, x + 1, y) - 2u(t, x, y) + u(t, x - 1, y)}{dx^2}$$

- Forward finite difference in time, central finite difference in space.

5-point stencil: the heat program

- Given an initial value of u , and any boundary conditions, we can calculate the value of u at time $t+1$ given the value at time t .
- Each update requires values from the north, south, east and west neighbours only:



- Computation is essentially a weighted average of each cell and its neighbouring cells.
- If on a boundary, look up a boundary condition instead.

Heat diffusion problem: 5-point stencil code

```
const double r = alpha * dt / (dx * dx);
const double r2 = 1.0 - 4.0*r;
```

```
// malloc and initialize u_tmp and u (code not shown)
```

```
for (int t = 0; t < nsteps; ++t) {
```

Loop over time steps

```
    for (int i = 0; i < n; ++i) {
```

```
        for (int j = 0; j < n; ++j) {
```

Loop over NxN spatial domain

```
            u_tmp[i+j*n] = r2 * u[i+j*n] +
                r * ((i < n-1) ? u[i+1+j*n] : 0.0) +
                r * ((i > 0) ? u[i-1+j*n] : 0.0) +
                r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
                r * ((j > 0) ? u[i+(j-1)*n] : 0.0);
```

Update the 5-point stencil. Boundary conditions on the edges of the domain are fixed at zero.

```
        }
```

```
    }
```

```
    // Pointer swap to get ready for next step
```

```
    tmp = u;
```

```
    u = u_tmp;
```

```
    u_tmp = tmp;
```

```
}
```

```
}
```

Heat diffusion problem: 5-point stencil code

```
const double r = alpha * dt / (dx * dx);
const double r2 = 1.0 - 4.0*r;
```

```
// malloc and initialize u_tmp and u (code not shown)
```

```
for (int t = 0; t < nsteps; ++t) {
```

```
    #pragma omp parallel for collapse(2)
```

```
    for (int i = 0; i < n; ++i) {
```

```
        for (int j = 0; j < n; ++j) {
```

```
            u_tmp[i+j*n] = r2 * u[i+j*n] +
                r * ((i < n-1) ? u[i+1+j*n] : 0.0) +
                r * ((i > 0) ? u[i-1+j*n] : 0.0) +
                r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
                r * ((j > 0) ? u[i+(j-1)*n] : 0.0);
```

```
        }
```

```
    }
```

```
    // Pointer swap to get ready for next step
```

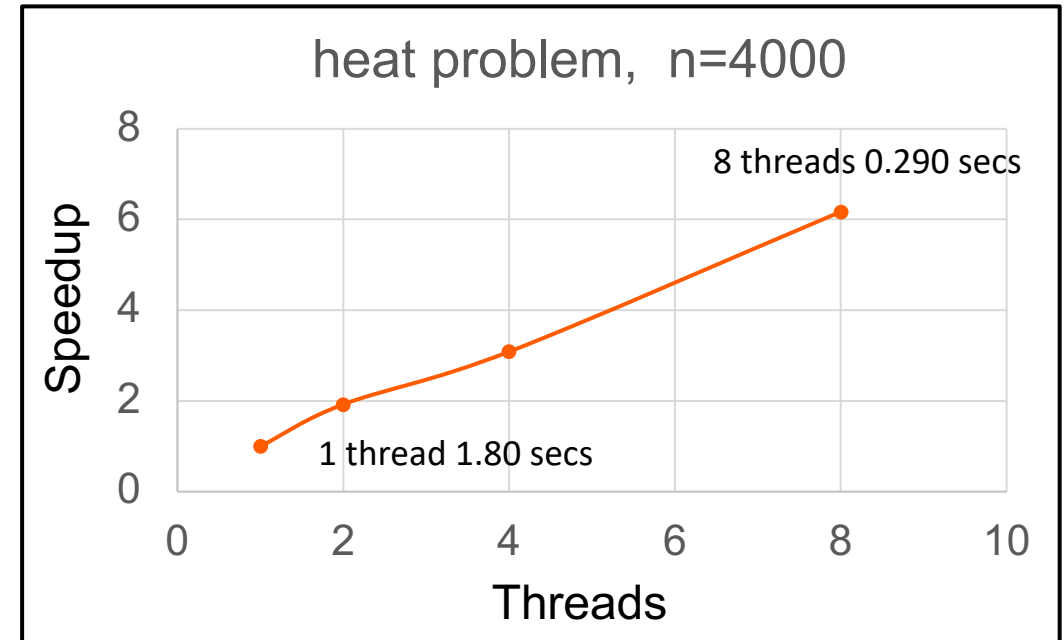
```
    tmp = u;
```

```
    u = u_tmp;
```

```
    u_tmp = tmp;
```

```
}
```

```
}
```



Intel® Xeon™ Gold 5218 @ 2.3 Ghz, 8 cores. Nvidia HPC Toolkit compiler
nvc -fast -fopenmp heat.c

Heat diffusion problem: 5-point stencil code

```

const double r = alpha * dt / (dx * dx);
const double r2 = 1.0 - 4.0*r;

// malloc and initialize u_tmp and u (code not shown)

for (int t = 0; t < nsteps; ++t) {
    #pragma omp target map(tofrom: u[0:n*n], u_tmp[0:n*n])
    #pragma omp loop
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            u_tmp[i+j*n] = r2 * u[i+j*n]
                           +
                           r * ((i < n-1) ? u[i+1+j*n] : 0.0) +
                           r * ((i > 0) ? u[i-1+j*n] : 0.0) +
                           r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
                           r * ((j > 0) ? u[i+(j-1)*n] : 0.0);
        }
    }
    // Pointer swap to get ready for next step
    tmp = u;
    u = u_tmp;
    u_tmp = tmp;
}
}

```

When you map pointers between the host and the device, OpenMP remembers the address.

Swapped addresses on the hosts swaps
addresses on the device

GPU Solver time = 1.40 secs

This isn't much better than the
runtime for a single CPU (1.8 secs)
and worse than 8 cores on a CPU
(0.29 secs).

Why is the performance so bad?

NVIDIA T4 GPU, 16 Gbyte, Turing Arch.
Nvidia HPC Toolkit compiler
nvc -fast -mp=gpu -gpu=cc75 heat.c

Heat diffusion problem: 5-point stencil code

```
const double r = alpha * dt / (dx * dx);
const double r2 = 1.0 - 4.0*r;

// malloc and initialize u_tmp and u (code not shown)
```

```
for (int t = 0; t < nsteps; ++t) {
    #pragma omp target map(tofrom: u[0:n*n], u_tmp[0:n*n])
    #pragma omp loop
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            u_tmp[i+j*n] = r2 * u[i+j*n]
                           +
                           r * ((i < n-1) ? u[i+1+j*n] : 0.0) +
                           r * ((i > 0) ? u[i-1+j*n] : 0.0) +
                           r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
                           r * ((j > 0) ? u[i+(j-1)*n] : 0.0);
        }
    }
    // Pointer swap to get ready for next step
    tmp = u;
    u = u_tmp;
    u_tmp = tmp;
}
}
```

With a runtime of 1.4 secs (worse than the CPU time) we see that Data Movement dominates performance.

At the beginning of each iteration, **copy** $(2*N^2)*\text{sizeof}(\text{TYPE})$ bytes **to** the device

We need to create a **data region** on the GPU that is distinct from the target region.

That way, we can keep the data on the device between target constructs

At the end of each iteration, **copy** $(2*N^2)*\text{sizeof}(\text{TYPE})$ bytes **from** the device

GPU data environments distinct from kernel execution

- Create a data region on the target device (a device data environment) with two standalone directives:

```
#pragma omp target enter data map(...)  
#pragma omp target exit data map(...)
```

- The **target enter data** maps variables to the device data environment.
- The **target exit data** unmaps variables from the device data environment.
- Once created, subsequent **target** regions inherit the existing data environment.

Heat diffusion problem: 5-point stencil code

```

const double r = alpha * dt / (dx * dx);
const double r2 = 1.0 - 4.0*r;
// malloc and initialize u_tmp and u (code not shown)
#pragma omp target enter data map(to: u[0:n*n], u_tmp[0:n*n])

for (int t = 0; t < nsteps; ++t) {
    #pragma omp target
    #pragma omp loop
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            u_tmp[i+j*n] = r2 * u[i+j*n] +
                r * ((i < n-1) ? u[i+1+j*n] : 0.0) +
                r * ((i > 0) ? u[i-1+j*n] : 0.0) +
                r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
                r * ((j > 0) ? u[i+(j-1)*n] : 0.0);
        }
    }
    // Pointer swap to get ready for next step
    tmp = u;
    u = u_tmp;
    u_tmp = tmp;
}
#pragma omp target exit data map(from: u[0:n*n])
}

```

Create a data region and map indicated data on entry

GPU Solver time* = **0.42** secs

This is a general principal ...
if you want performance, you
must optimize data
movement.

*includes time for target enter/exit data

Exit the data
region and map
indicated data

NVIDIA T4 GPU, 16 Gbyte, Turing Arch.
Nvidia HPC Toolkit compiler
nvc -fast -mp=gpu -gpu=cc75 heat.c

Optimizing our 5-point stencil code

Parallel CPU and GPU results, n=4000

```
const double r = alpha * dt / (dx * dx);
const double r2 = 1.0 - 4.0*r;
// malloc and initialize u_tmp and u (code not shown)
```

```
for (int t = 0; t < nsteps; ++t) {
```

```
    #pragma omp parallel for
```

```
    for (int i = 0; i < n; ++i) {
```

```
        for (int j = 0; j < n; ++j) {
```

```
            u_tmp[i+j*n] = r2 * u[i+j*n]
                + r * ((i < n-1) ? u[i+1+j*n] : 0.0) +
                r * ((i > 0) ? u[i-1+j*n] : 0.0) +
                r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
                r * ((j > 0) ? u[i+(j-1)*n] : 0.0);
```

```
        }
```

```
    }
```

```
    // Pointer swap for next step
```

```
    tmp = u;
```

```
    u = u_tmp;
```

```
    u_tmp = tmp;
```

```
}
```

```
}
```

This is the ij loop order. Swap these loops to get the ji order.

What if I optimize the CPU code?

... just swap the i and j loops. Using collapse(2) did not help on the GPU or the CPU

CPU	Num threads	ij loop order	ji loop order
	1	1.512849	0.262260
	2	0.776229	0.132453
	4	0.400822	0.064220
	8	0.227317	0.046586

Intel® Xeon™ Gold 5218 @ 2.3 Ghz, 8 cores.
Nvidia HPC Toolkit compiler nvc -fast -fopenmp heat.c

All times in seconds

GPU	ij without timing enter and exit data	ij loop order	ji without timing enter and exit data	ji loop order
	0.056830	0.417887	0.020123	0.358905

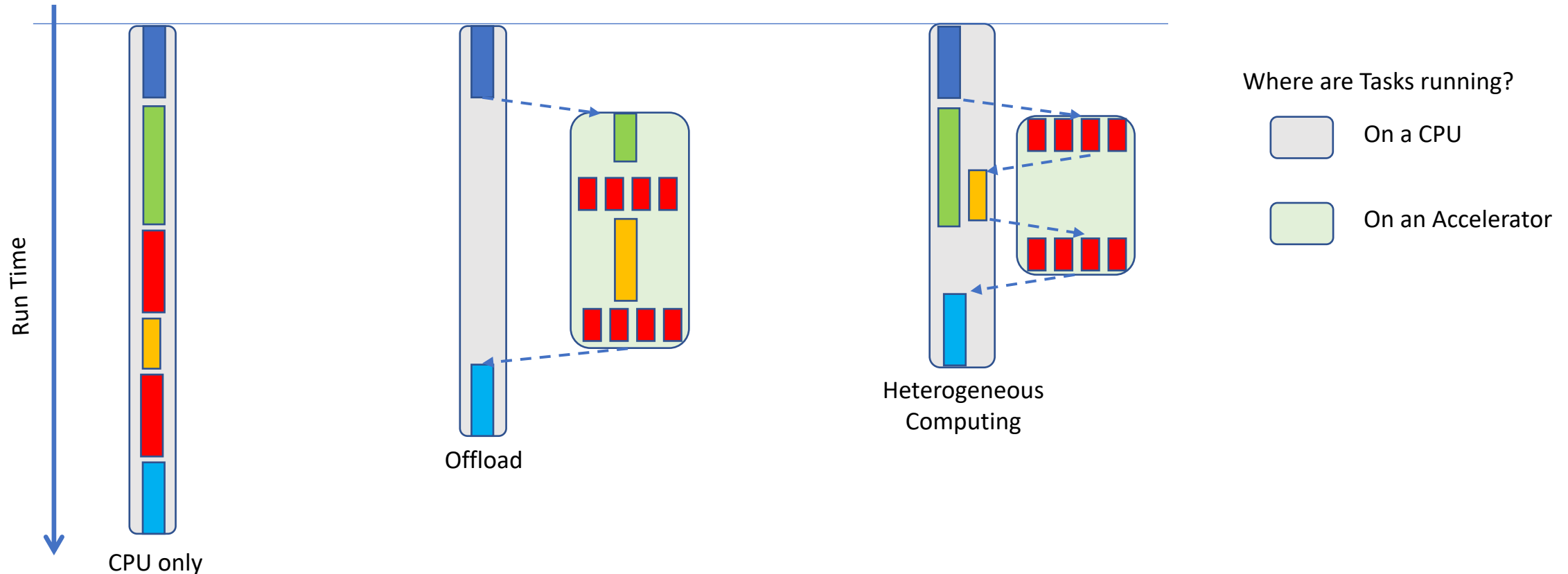
This GPU code used the target enter data and target exit data

NVIDIA T4 GPU, 16 Gbyte, Turing Arch.
Nvidia HPC Toolkit compiler. nvc -fast -mp=gpu heat.c

Why is OpenMP so important for GPU programming?

No single processor is best at everything

- The idea that you should move everything to the GPU makes no sense
- **Heterogeneous Computing:** Run sub-problems in parallel on the hardware best suited to them.



OpenMP: the only option for mixing CPU and GPU programming in a single program

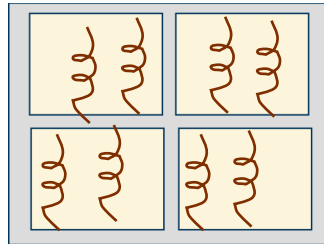
```
int main()
{
    float a[N], b[N], c[N], res[N];
    int err=0;

    // fill the arrays
    #pragma omp parallel for
    for (int i=0; i<N; i++){
        a[i] = (float)i;
        b[i] = 2.0*(float)i;
        c[i] = 0.0;
        res[i] = i + 2*i;
    }

    // add two vectors
    #pragma omp target
    #pragma omp loop
    for (int i=0; i<N; i++){
        c[i] = a[i] + b[i];
    }

    // test results
    #pragma omp parallel for reduction(+:err)
    for(int i=0; i<N; i++){
        float val = c[i] - res[i];
        val = val*val;
        if(val>TOL) err++;
    }
    printf("vectors added with %d errors\n", err);
    return 0;
}
```

Hardware is diverse ... and its only getting worse!!!



CPU

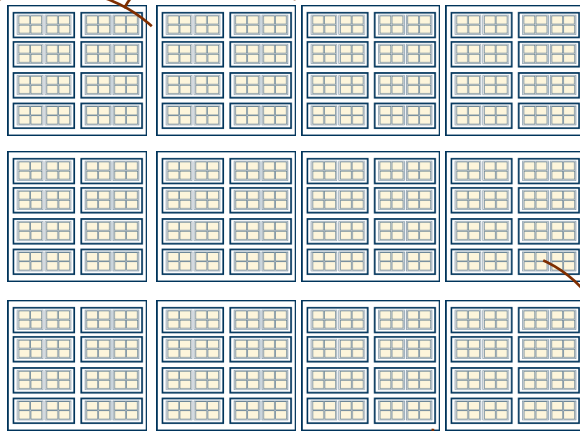


SIMD/Vector

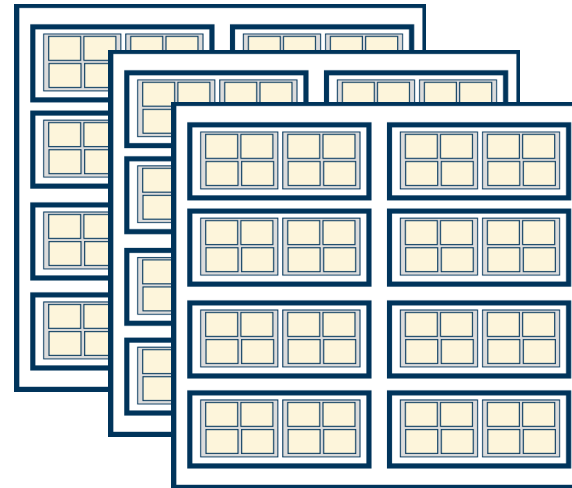


GPU

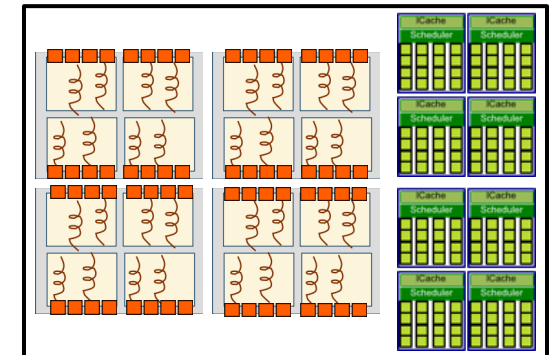
We'll now consider parallel programming with distributed memories



Cloud



Cluster



Heterogeneous node

Parallel API's: MPI ... the Message Passing Interface

MPI: An API for Writing Applications for Distributed memory systems

- A library of routines to coordinate the execution of multiple processes.
- Provides point to point and collective communication in Fortran, C and C++
- Unifies last 35+ years of cluster computing and MPP practice

MPI_Type_contiguous

MPI_Recv_init

MPI_Scan

MPI_Allgather

MPI_Start

MPI_Bsend_init

MPI_Sendrecv_replace

MPI_Alltoallv

MPI_Send

MPI_Bcast

MPI_Type_free

MPI_Unpack

CSOMP_ORDERED

MPI_Reduce

MPI_Bsend_init

MPI_Recv

MPI_Test_cancelled

MPI_Ssend

MPI_Waitall

MPI_Recv_init

MPI_Group_size

MPI_COMM_WORLD

MPI_Group_compare

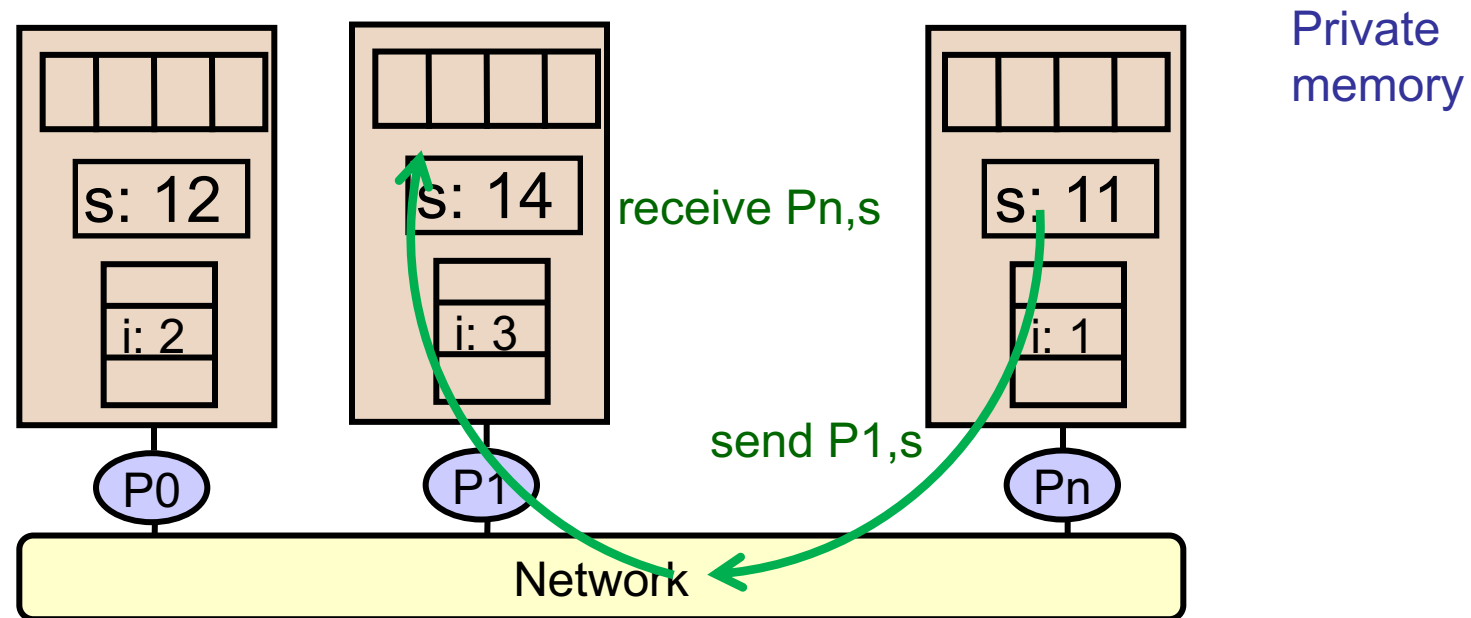
MPI_Startall

MPI_Pack

omp_set_lock(lck)

Programming Model: Message Passing

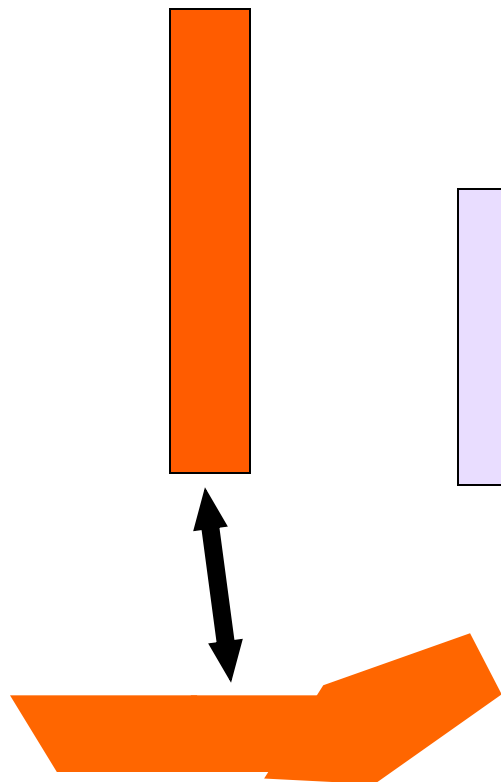
- Program consists of a collection of processes.
 - Number of processes almost always fixed at program startup time
 - Local address space per node -- NO physically shared memory.
 - Logically shared data is partitioned over local processes.
- Processes communicate by explicit send/receive pairs
 - Synchronization is implicit by communication events.
 - MPI (Message Passing Interface) is the most commonly used API



How do people use MPI?

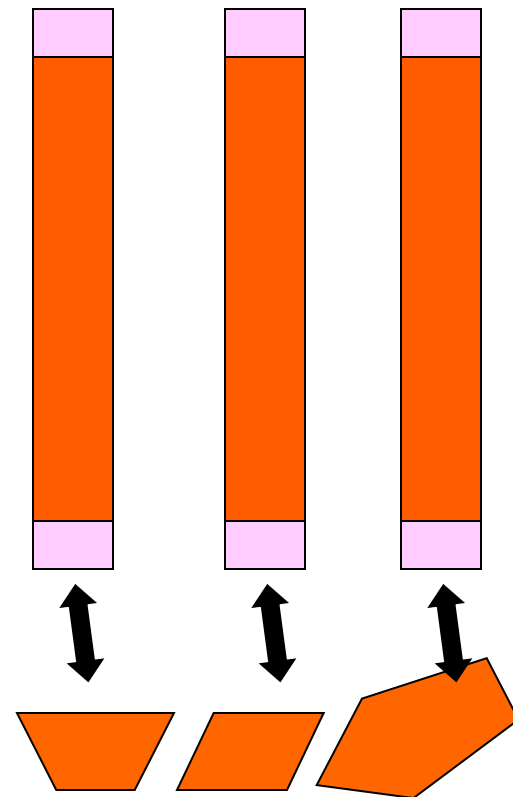
The SPMD Design Pattern

A sequential program
working on a data set



Replicate the program.
Add glue code
Break up the data

- A single program working on a decomposed data set.
- Use Node ID and numb of nodes to split up work between processes
- Coordination by passing messages.



MPI Hello World Program

```
#include <stdio.h>
#include <mpi.h>
int main (int argc, char **argv){
    int rank, size;
    MPI_Init (&argc, &argv);
    MPI_Comm_rank (MPI_COMM_WORLD, &rank);
    MPI_Comm_size (MPI_COMM_WORLD, &size);
    printf( "Hello from process %d of %d\n",
            rank, size );

    MPI_Finalize();
    return 0;
}
```

MPI Hello World Program

```
int MPI_Init (int* argc, char* argv[])
```

- Initializes the MPI library ... called before any other MPI functions.
- argc and argv are the command line args passed from main()

```
#include <stdio.h>
#include <mpi.h>
int main (int argc, char **argv){
    int rank, size;
    MPI_Init (&argc, &argv);
    MPI_Comm_rank (MPI_COMM_WORLD, &rank);
    MPI_Comm_size (MPI_COMM_WORLD, &size);
    printf( "Hello from process %d of %d\n",
            rank, size );

    MPI_Finalize();
    return 0;
}
```

```
int MPI_Finalize (void)
```

- Frees memory allocated by the MPI library ... close every MPI program with a call to MPI_Finalize

MPI Hello World Program

```
int MPI_Comm_size (MPI_Comm comm, int* size)
```

- `MPI_Comm`, an *opaque data type called a communicator*.
Default context: `MPI_COMM_WORLD` (all processes)
- `MPI_Comm_size` returns the number of processes in the process group associated with the communicator

```
#include <stdio.h>
#include <mpi.h>
int main (int argc, char **argv){
    int rank, size;
    MPI_Init (&argc, &argv);
    MPI_Comm_rank (MPI_COMM_WORLD, &rank);
    MPI_Comm_size (MPI_COMM_WORLD, &size);
    printf( "Hello from process %d of %d\n",
            rank, size );

    MPI_Finalize();
    return 0;
}
```

Communicators consist of two parts, a **context** and a **process group**.

The communicator lets one control how groups of messages interact.

Communicators support modular SW ... i.e. I can give a library module its own communicator and know that its messages can't collide with messages originating from outside the module

MPI Hello World Program

```
int MPI_Comm_rank (MPI_Comm comm, int* rank)
```

- `MPI_Comm`, an *opaque data type*, a communicator. Default context: `MPI_COMM_WORLD` (all processes)
- `MPI_Comm_rank` An integer ranging from 0 to “(num of procs)-1”

```
#include <stdio.h>
#include <mpi.h>
int main (int argc, char **argv){
    int rank, size;
    MPI_Init (&argc, &argv);
    MPI_Comm_rank (MPI_COMM_WORLD, &rank);
    MPI_Comm_size (MPI_COMM_WORLD, &size);
    printf( "Hello from process %d of %d\n",
            rank, size );

    MPI_Finalize();
    return 0;
}
```

Note that other than `init()` and `finalize()`, every MPI function has a communicator.

This makes sense .. You need a context and group of processes that the MPI functions impact ... and those come from the communicator.

Bulk Synchronous Programming (BSP):

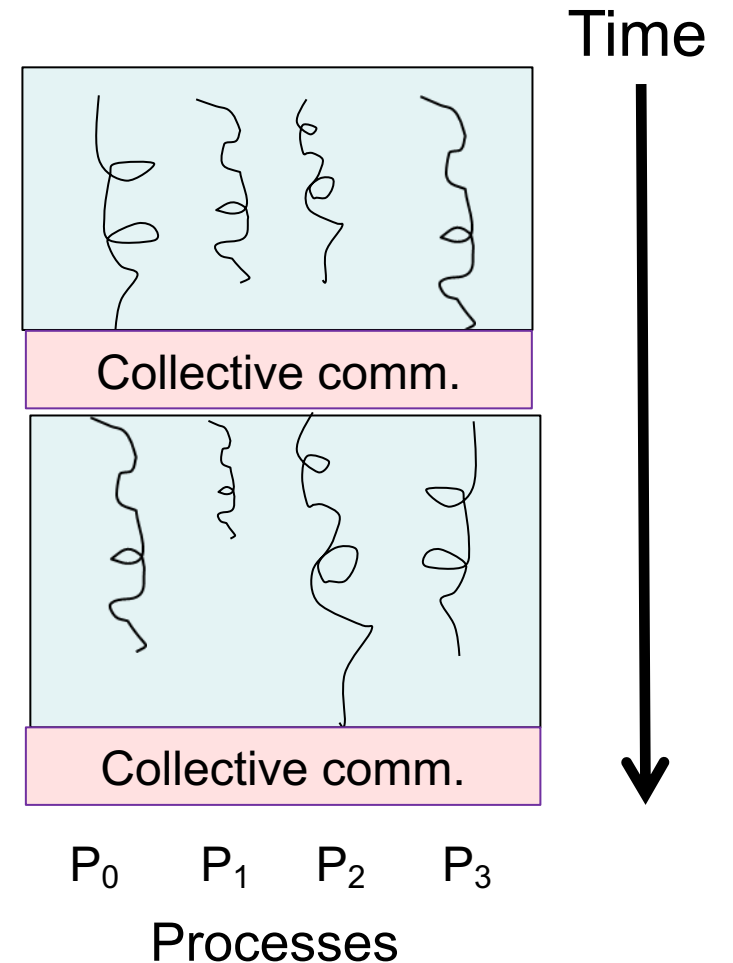
A common design pattern used with MPI Programs

BSP includes the map-reduce pattern commonly used in the cloud. It's easy to use and surprisingly useful

- Many MPI applications have few (if any) sends and receives.

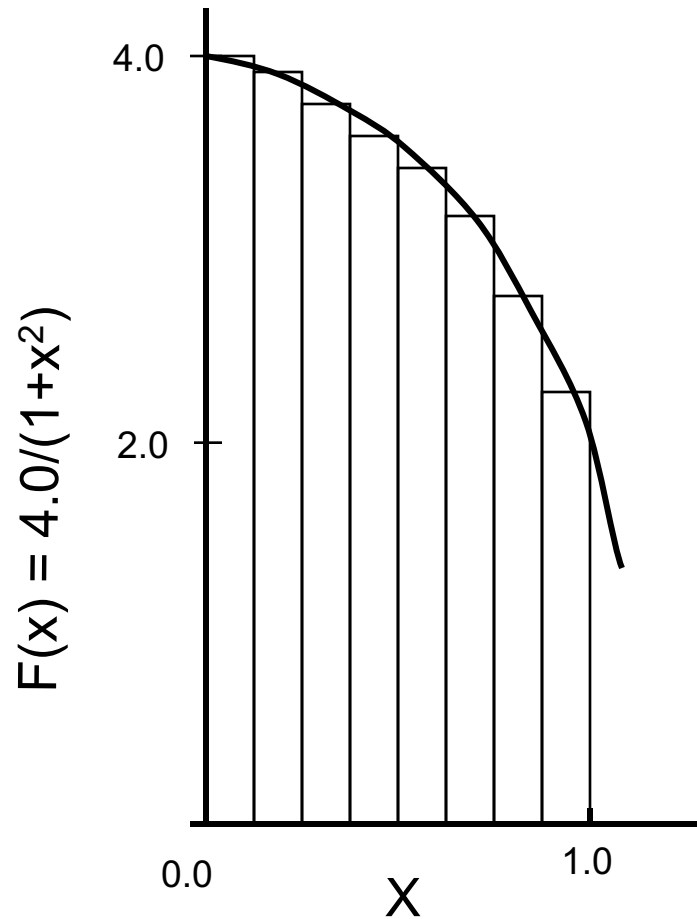
They use the following very common pattern:

- Use the Single Program Multiple Data pattern
- Each process maintains a local view of the global data
- A problem broken down into phases each of which is composed of two subphases:
 - Compute on local view of data
 - Communicate to update global view on all processes (collective communication).
- Continue phases until complete



This is a subset of the SPMD pattern sometimes referred to as the Bulk Synchronous pattern.

Example Problem: Numerical Integration



Mathematically, we know that:

$$\int_0^1 \frac{4.0}{(1+x^2)} dx = \pi$$

We can approximate the integral as a sum of rectangles:

$$\sum_{i=0}^N F(x_i) \Delta x \approx \pi$$

Where each rectangle has width Δx and height $F(x_i)$ at the middle of interval i .

PI Program: an example

```
static long num_steps = 100000;
double step;
void main ()
{   int i;   double x, pi, sum = 0.0;

    step = 1.0/(double) num_steps;
    x = 0.5 * step;
    for (i=0;i<= num_steps; i++){
        x+=step;
        sum += 4.0/(1.0+x*x);
    }
    pi = step * sum;
}
```

Pi program in MPI

```
#include <mpi.h>
```

```
void main (int argc, char *argv[])
```

```
{
```

```
    int i, my_id, numprocs; double x, pi, step, sum = 0.0 ;
```

```
    step = 1.0/(double) num_steps ;
```

```
    MPI_Init(&argc, &argv) ;
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &my_id) ;
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &numprocs) ;
```

```
    my_steps = num_steps/numprocs ;
```

```
    for (i=my_id; i<num_steps; i+=numprocs)
```

```
    {
```

```
        x = (i+0.5)*step;
```

```
        sum += 4.0/(1.0+x*x);
```

```
    }
```

```
    sum *= step ;
```

```
    MPI_Reduce(&sum, &pi, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD); ;
```

```
}
```



Sum values in "sum" from
each process and place it
in "pi" on process 0

MPI Pi program performance (on my laptop)

```
#include <mpi.h>
```

```
void main (int argc, char *argv[])
```

```
{
```

```
    int i, my_id, numprocs; double x, pi, step, sum = 0.0 ;
```

```
    step = 1.0/(double) num_steps ;
```

```
    MPI_Init(&argc, &argv) ;
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &my_id) ;
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &numprocs) ;
```

```
    double init_time = MPI_Wtime();
```

```
    my_steps = num_steps/numprocs ;
```

```
    for (i=my_id*my_steps; i<(my_id+1)*my_steps ; i++)
```

```
    {
```

```
        x = (i+0.5)*step;
```

```
        sum += 4.0/(1.0+x*x);
```

```
    }
```

```
    sum *= step ;
```

```
    MPI_Reduce(&sum, &pi, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
```

```
    if(my_id == 0) printf(" runtime = %lf\n",MPI_Wtime()-init_time);
```

```
}
```

Thread or procs	OpenMP SPMD critical	OpenMP PI Loop	MPI
1	0.85	0.43	0.84
2	0.48	0.23	0.48
3	0.47	0.23	0.46
4	0.46	0.23	0.46

*Intel compiler (icpc) with -O3 on Apple OS X 10.7.3 with a dual core (four HW thread)
Intel® Core™ i5 processor at 1.7 Ghz and 4 Gbyte DDR3 memory at 1.333 Ghz.

Message passing

Message passing: Basic ideas and jargon

- We need to coordinate the execution of processes ... which may be spread out over a collection of independent computers
- Coordination:
 1. Process management (e.g., create and destroy)
 2. Synchronization ... timing constraints for concurrent processes)
 3. Communication ... Passing a buffer from one machine to another
- A message passing interface builds coordination around messages (either explicitly or implicitly).
- The fundamental (and overly simple) timing model for a message:

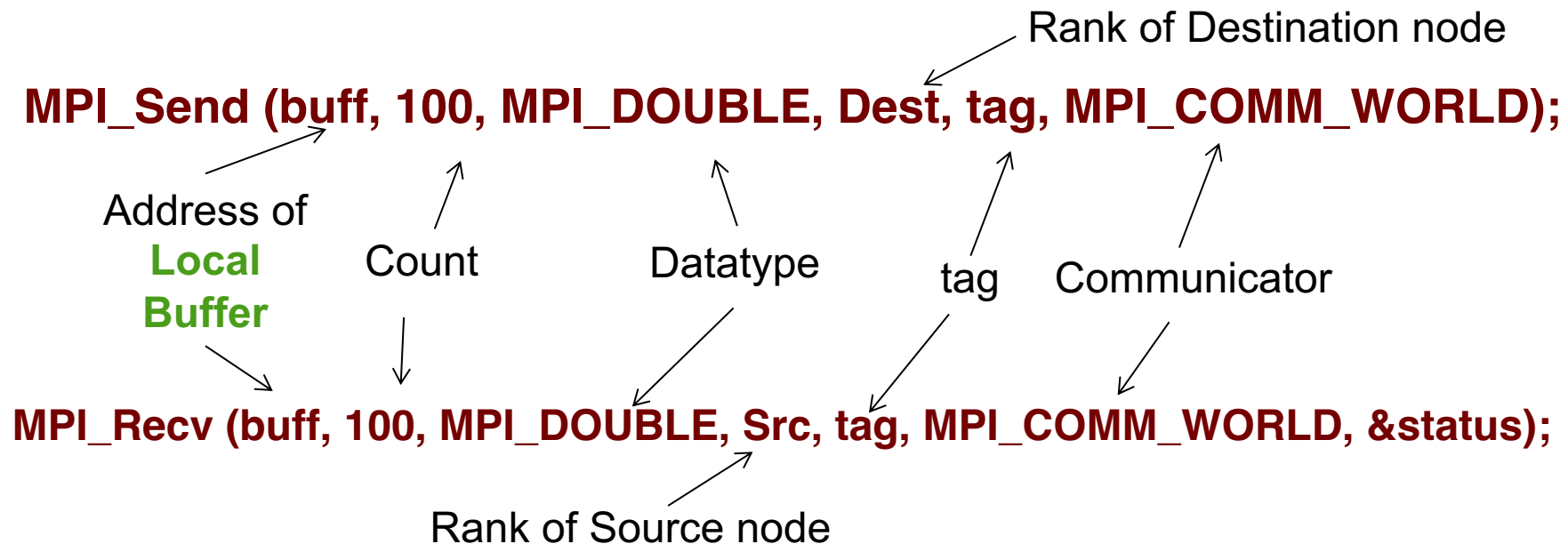
$$\text{Time}_{\text{communication}} = \text{latency} + N_{\text{bytes}} / \text{bandwidth}$$

Network fixed costs plus overheads

Network asymptotic bytes per second

Sending and receiving messages

- Pass a buffer which holds “count” values of MPI_TYPE
- The data in a message to send or receive is described by a triple:
 - **(address, count, datatype)**
- The receiving process identifies messages with the double :
 - **(source, tag)**
- Where:
 - Source is the rank of the sending process
 - Tag: a user-defined int to keep track of different messages from a single source



Solution: Ping-Pong Program

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#define VAL 42
#define NREPS 10
#define TAG 5

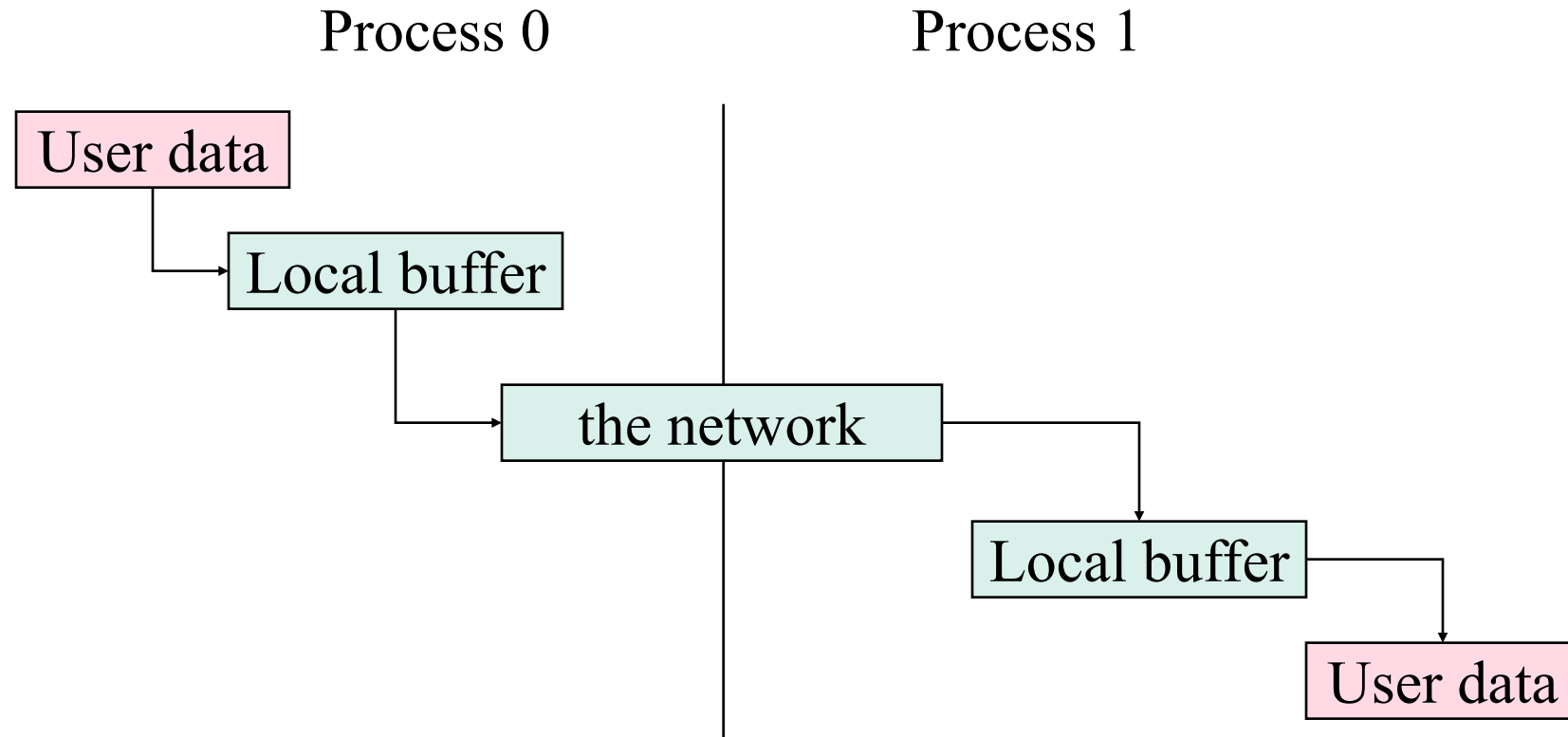
int main(int argc, char **argv) {
    int rank, size;
    double t0;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int bsend = VAL;
    int brecv = 0;
    MPI_Status stat;
    MPI_Barrier(MPI_COMM_WORLD);
    if(rank == 0) t0 = MPI_Wtime();
```

```
    for(int i=0;i<NREPS; i++){
        if(rank == 0){
            MPI_Send(&bsend, 1, MPI_INT, 1, TAG, MPI_COMM_WORLD);
            MPI_Recv(&brecv, 1, MPI_INT, 1, TAG, MPI_COMM_WORLD, &stat);
            if(brecv != VAL)printf("error: iteration %d %d != %d\n",i,brecv,VAL);
            brecv = 0;
        }
        else if(rank == 1){
            MPI_Recv(&brecv, 1, MPI_INT, 0, TAG, MPI_COMM_WORLD, &stat);
            MPI_Send(&bsend, 1, MPI_INT, 0, TAG, MPI_COMM_WORLD);
            if(brecv != VAL)printf("error: iteration %d %d != %d\n",i,brecv,VAL);
            brecv = 0;
        }
    }
    if(rank == 0){
        double t = MPI_Wtime() - t0;
        double lat = t/(2*NREPS);
        printf(" lat = %f seconds\n",(float)lat);
    }
    MPI_Finalize();
}
```

Buffers

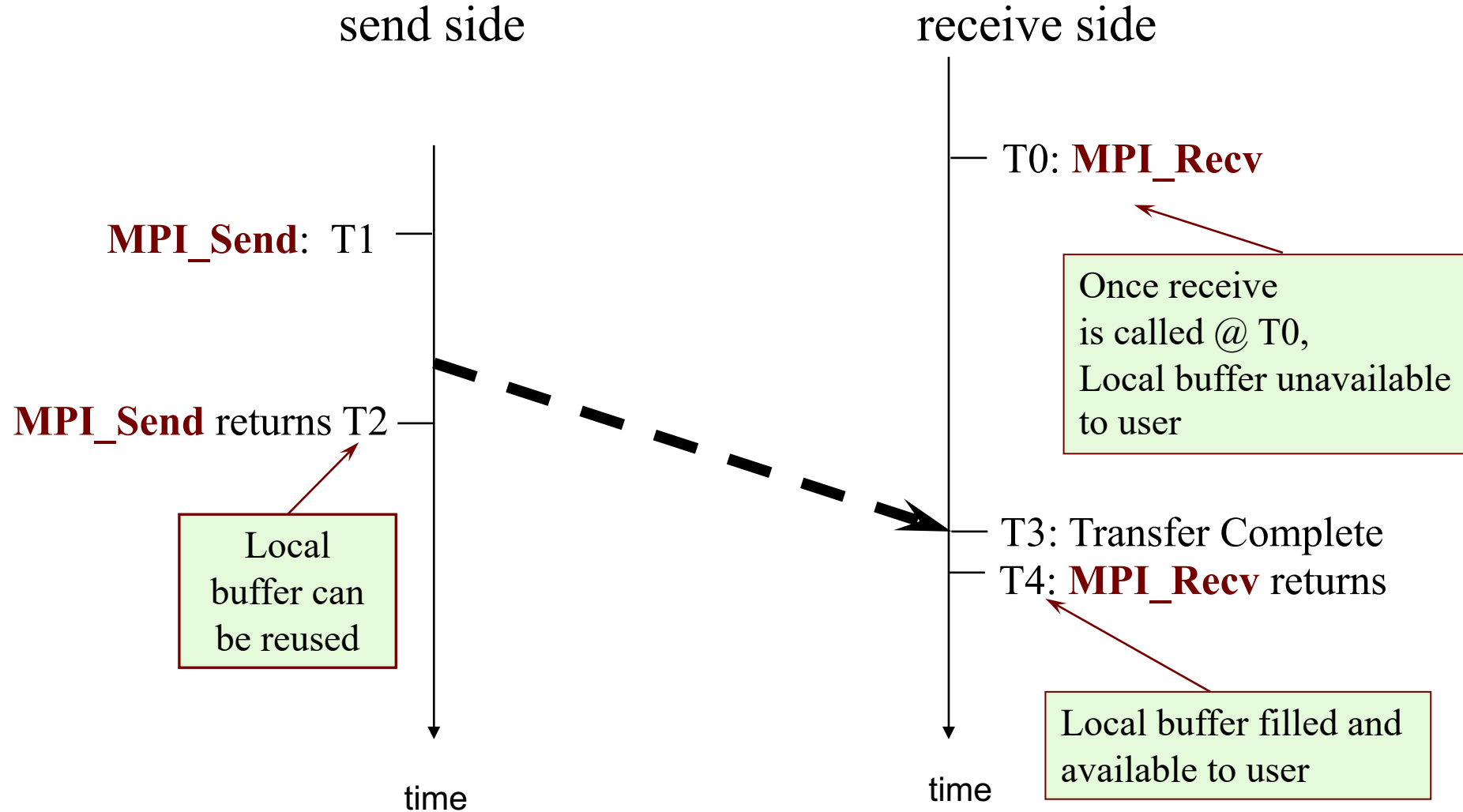
- Message passing is straightforward, but there are subtleties
 - Buffering and deadlock
 - Deterministic execution
 - Performance
- When you send data, where does it go? The following is the typical flow:



Derived from slides provided by Bill Gropp of UIUC

Blocking Send-Receive Timing Diagram

(Receive before Send)



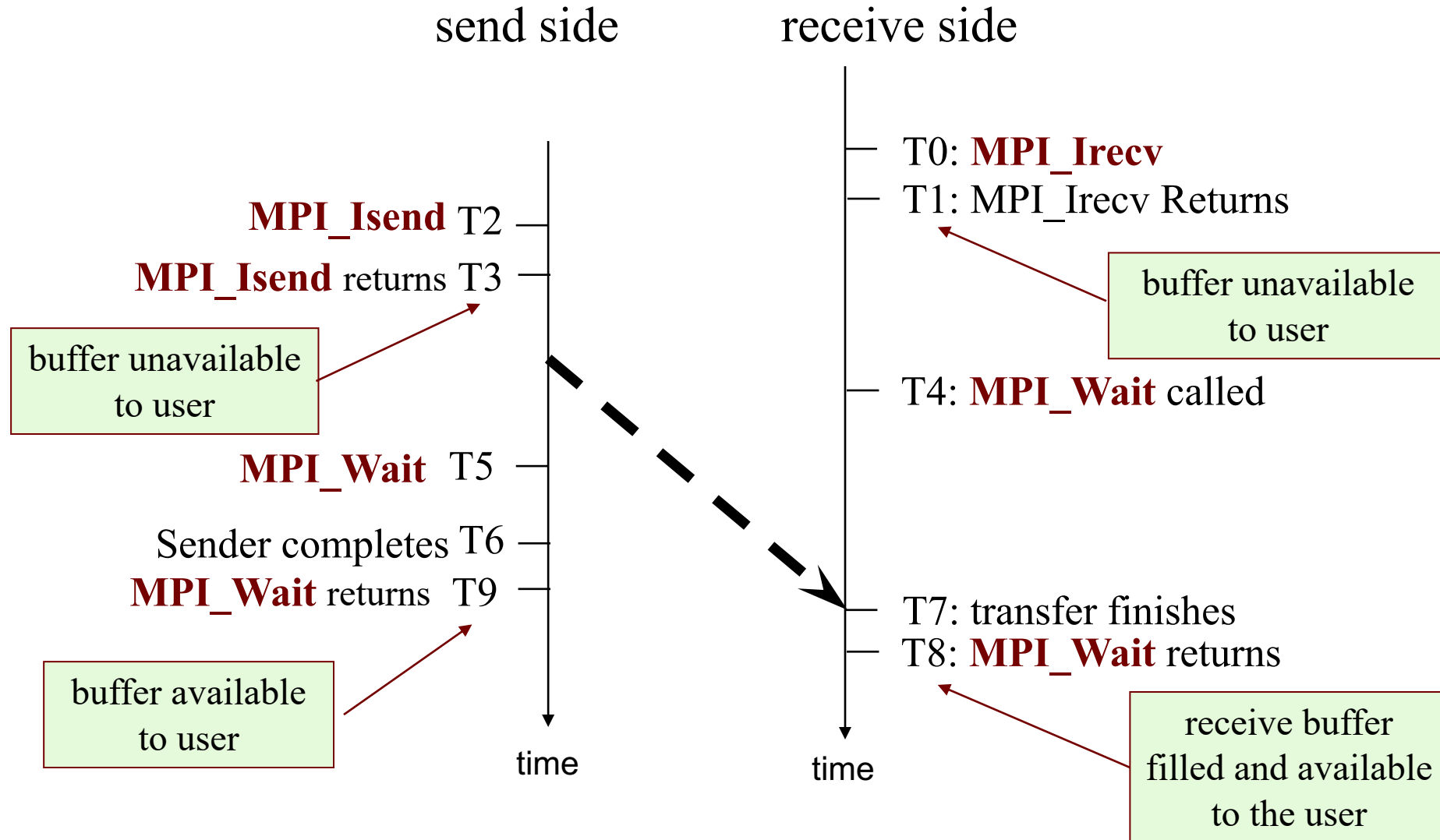
It is important to post the receive before sending, for highest performance.

Non-Blocking Communication

- Non-blocking operations return immediately and pass “request handles” that can be waited on and queried
 - **MPI_Isend(start, count, datatype, dest, tag, comm, request)**
 - **MPI_Irecv(start, count, datatype, src, tag, comm, request)**
 - **MPI_Wait(request, status)**
- One can also test without waiting using **MPI_TEST**
 - **MPI_Test(request, flag, status)**
- Anywhere you use **MPI_Send** or **MPI_Recv**, you can use the pair of **MPI_Isend/MPI_Wait** or **MPI_Irecv/MPI_Wait**
- Note the MPI types:
 - MPI_Status status;** // type used with the status output from recv
 - MPI_Request request;** // the type of the handle used with isend/ircv

Non-blocking operations are extremely important ... they allow you to overlap computation and communication.

Non-Blocking Send-Receive Diagram



Example: shift messages around a ring

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char **argv)
{
    int num, rank, size, tag, next, from;
    MPI_Status status1, status2;
    MPI_Request req1, req2;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank( MPI_COMM_WORLD, &rank);
    MPI_Comm_size( MPI_COMM_WORLD, &size);
    tag = 201;
    next = (rank+1) % size;
    from = (rank + size - 1) % size;
    if (rank == 0) {
        printf("Enter the number of times around the ring: ");
        scanf("%d", &num);

        printf("Process %d sending %d to %d\n", rank, num, next);
        MPI_Isend(&num, 1, MPI_INT, next, tag,
                  MPI_COMM_WORLD, &req1);
        MPI_Wait(&req1, &status1);
    }
}
```

```
do {
    MPI_Irecv(&num, 1, MPI_INT, from, tag,
              MPI_COMM_WORLD, &req2);
    MPI_Wait(&req2, &status2);

    if (rank == 0) {
        num--;
        printf("Process 0 decremented number\n");
    }

    printf("Process %d sending %d to %d\n", rank, num, next);
    MPI_Isend(&num, 1, MPI_INT, next, tag,
              MPI_COMM_WORLD, &req1);
    MPI_Wait(&req1, &status1);
} while (num != 0);

if (rank == 0) {
    MPI_Irecv(&num, 1, MPI_INT, from, tag,
              MPI_COMM_WORLD, &req2);
    MPI_Wait(&req2, &status2);
}
MPI_Finalize();
return 0;
}
```

What message passing looks like in "real" programs

Example: finite difference methods

- Solve the heat diffusion equation in 1 D:
 - $u(x,t)$ describes the temperature field
 - We set the heat diffusion constant to one
 - Boundary conditions, constant u at endpoints.

$$\frac{\partial^2 u}{\partial x^2} = \frac{\partial u}{\partial t}$$

- map onto a mesh with stepsize h and k

$$x_i = x_0 + ih \quad t_i = t_0 + ik$$

- Central difference approximation for spatial derivative (at fixed time)

$$\frac{\partial^2 u}{\partial x^2} = \frac{u_{j+1} - 2u_j + u_{j-1}}{h^2}$$

- Time derivative at $t = t^{n+1}$

$$\frac{du}{dt} = \frac{u^{n+1} - u^n}{k}$$

Example: Explicit finite differences

- Combining time derivative expression using spatial derivative at $t = t_n$

$$\frac{u_j^{n+1} - u_j^n}{k} = \frac{u_{j+1}^n - 2u_j^n + u_{j-1}^n}{h^2}$$

- Solve for u at time $n+1$ and step j

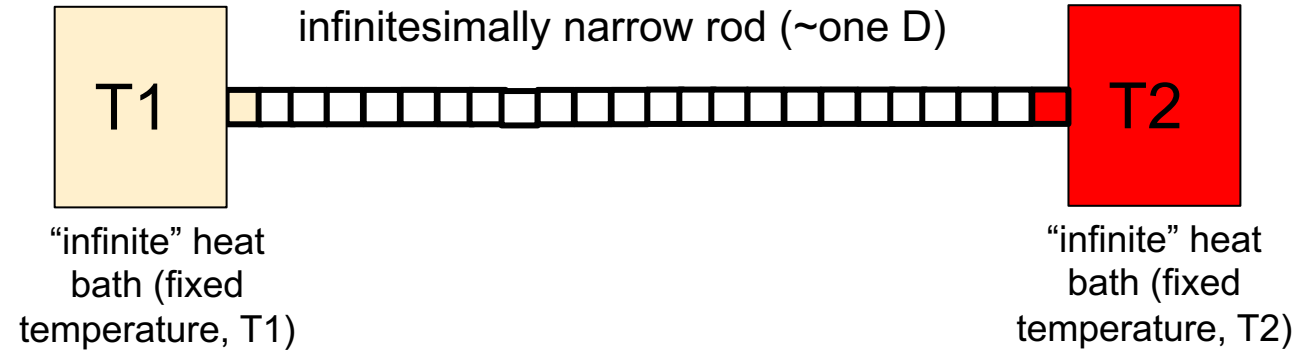
$$u_j^{n+1} = (1 - 2r)u_j^n + ru_{j-1}^n + ru_{j+1}^n \quad r = k/h^2$$

- The solution at $t = t_{n+1}$ is determined explicitly from the solution at $t = t_n$ (assume $u[t][0] = u[t][N] = \text{Constant}$ for all t).

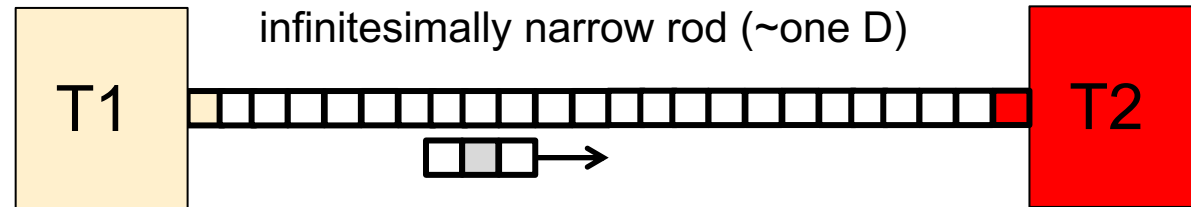
```
for (int t = 0; t < N_STEPS-1; ++t)
    for (int x = 1; x < N-1; ++x)
        u[t+1][x] = u[t][x] + r*(u[t][x+1] - 2*u[t][x] + u[t][x-1]);
```

- Explicit methods are easy to compute ... each point updated based on nearest neighbors. Converges for $r < 1/2$.

Heat Diffusion equation



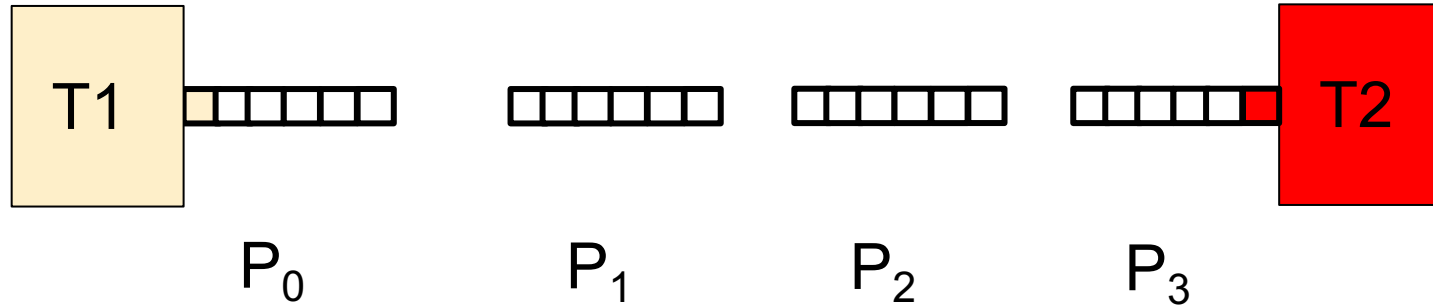
Heat Diffusion equation



Pictorially, you are sliding a three point "stencil" across the domain (u) and updating the center point at each stop.

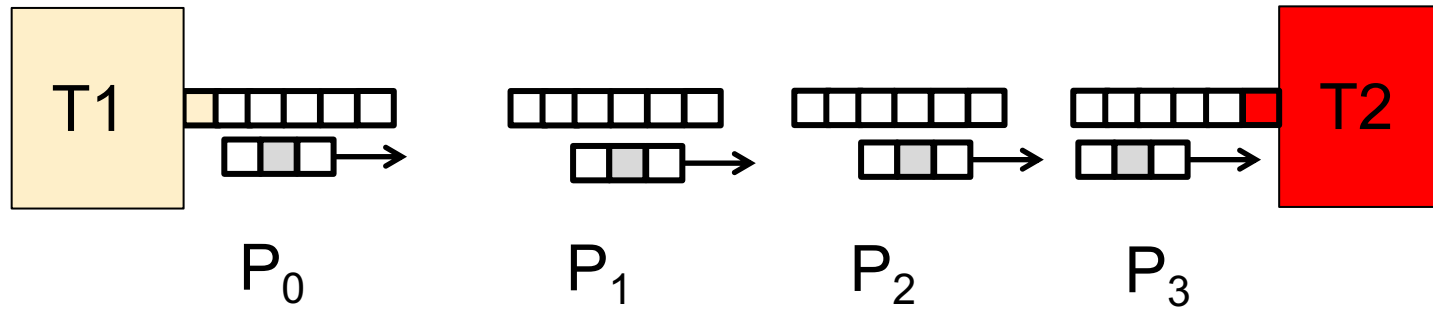
Heat Diffusion equation: in parallel

- Break it into chunks assigning one chunk to each process.



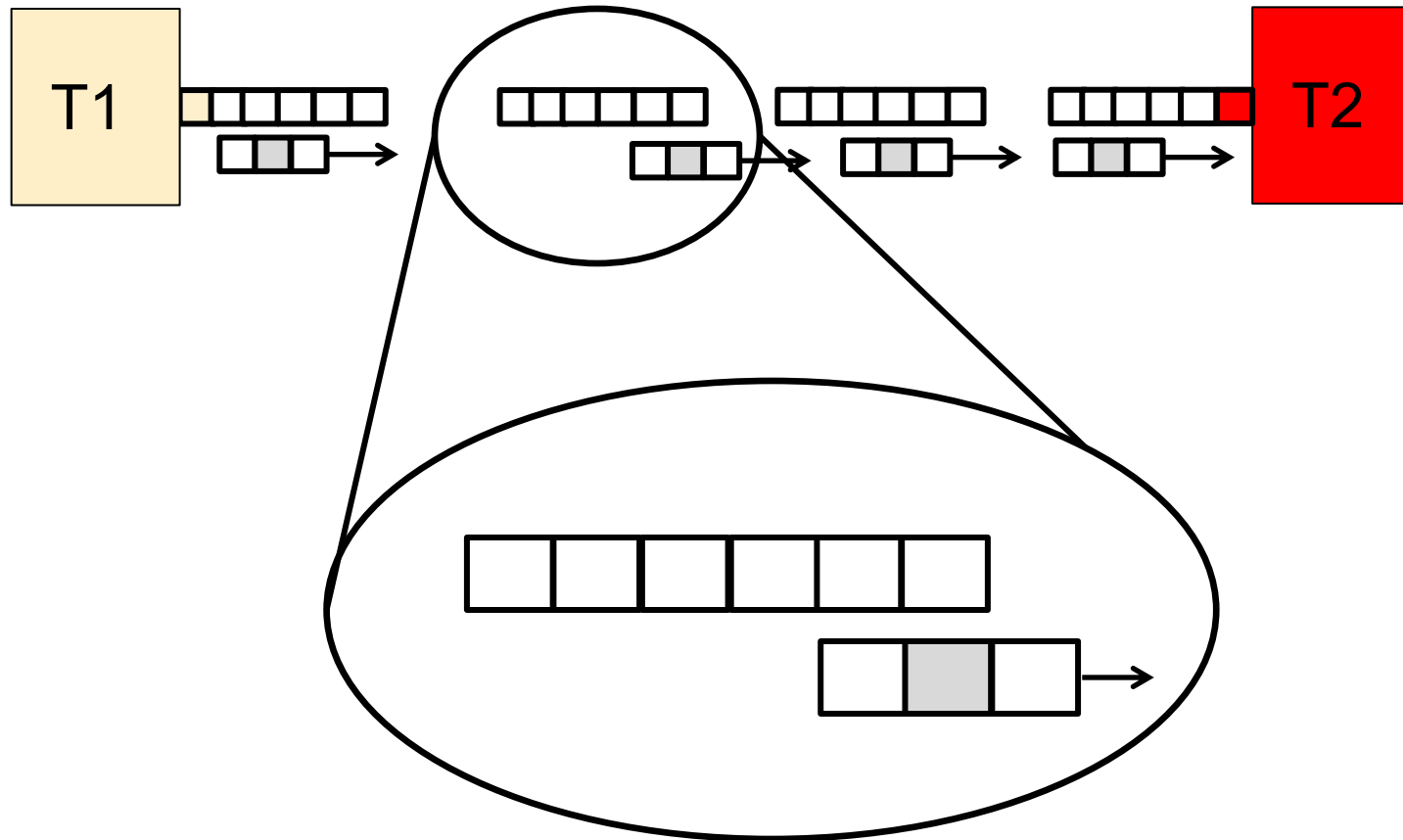
Heat Diffusion equation

- Each process works on its own chunk ... sliding the stencil across the domain to updates its own data.



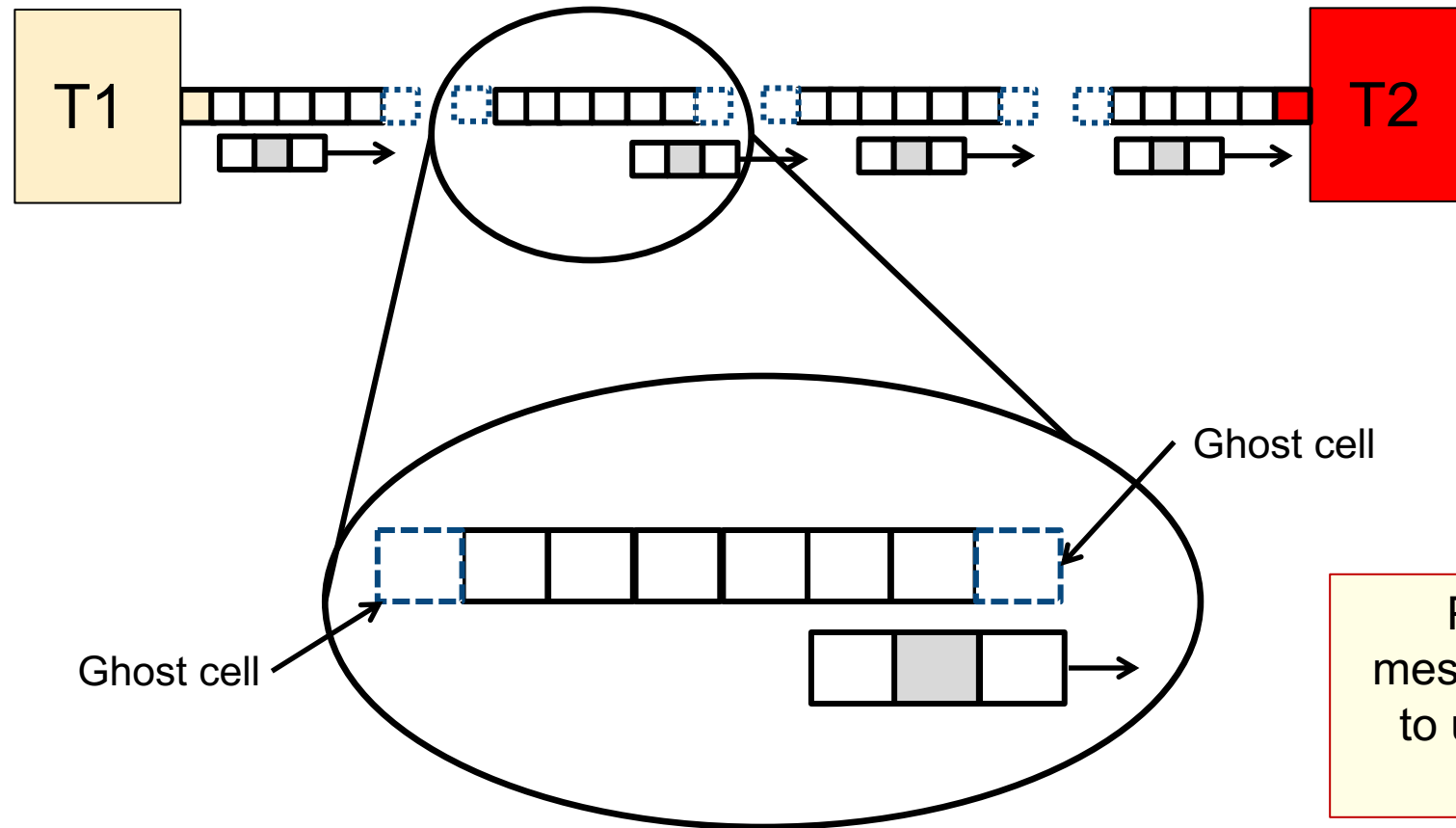
Heat Diffusion equation

- What about the ends of each chunk ... where the stencil runs off the end and has missing values for the computation?



Heat Diffusion equation

- We add ghost cells to the ends of each chunk, update them with the required values from neighbor chunks at each time step ... hence giving the stencil everything it needs on any given chunk to update all of its values.



Heat Diffusion MPI Example

```
MPI_Init (&argc, &argv);
MPI_Comm_size (MPI_COMM_WORLD, &P);
MPI_Comm_rank (MPI_COMM_WORLD, &myID);
double *u      = malloc (sizeof(double) * (2 + N/P)) // include "Ghost Cells" to hold
double *up1 = malloc (sizeof(double) * (2 + N/P)); // values from my neighbors

initialize_data(uk, ukp1, N, P);
for (int t = 0; t < N_STEPS; ++t){
    if (myID != 0) MPI_Send (&u[1], 1, MPI_DOUBLE, myID-1, 0, MPI_COMM_WORLD);
    if (myID != P-1) MPI_Recv (&u[N/P+1], 1, MPI_DOUBLE, myID+1, 0, MPI_COMM_WORLD, &status);
    if (myID != P-1) MPI_Send (&u[N/P], 1, MPI_DOUBLE, myID+1, 0, MPI_COMM_WORLD);
    if (myID != 0) MPI_Recv (&u[0], 1, MPI_DOUBLE, myID-1, 0, MPI_COMM_WORLD, &status);

    for (int x = 1; x <= N/P; ++x)
        up1[x] = u[x] + (k / (h*h)) * (u[x+1] - 2*u[x] + u[x-1]);
    if (myID != 0)
        up1[1] = u[1] + (k / (h*h)) * (u[1+1] - 2*u[1] + u[1-1]);
    if (myID != P-1)
        up1[N/P] = u[N/P] + (k/(h*h)) * (u[N/P+1] - 2*u[N/P] + u[N/P-1]);
    temp = up1; up1 = u; u = temp;
} // End of for (int t ...) loop

MPI_Finalize();
return 0;
```

**Exchange ghost
cells**

**Do the
computation, but
keep track of
boundary
conditions**

The code can get a bit ugly as each process figures out who to send to and who to receive from.

... but the concept is straightforward and eventually you get used to it.

The 12 core functions in MPI

- MPI_Init
- MPI_Finish
- MPI_Comm_size
- MPI_Comm_rank
- MPI_Send
- MPI_Recv
- MPI_Reduce
- MPI_Isend
- MPI_Irecv
- MPI_Wait
- MPI_Wtime
- MPI_Bcast

The ~~12~~ core functions in MPI

- MPI_Init
- MPI_Finish
- MPI_Comm_size
- MPI_Comm_rank
- ~~MPI_Send~~ → **Real Programmers always try to overlap communication and computation .. Post your receives using MPI_Irecv() then where appropriate, use MPI_Isend().**
- ~~MPI_Recv~~ →
- MPI_Reduce
- MPI_Isend
- MPI_Irecv
- MPI_Wait
- MPI_Wtime
- MPI_Bcast

The ~~12~~ core functions in MPI

- MPI_Init
- MPI_Finish
- MPI_Comm_size
- MPI_Comm_rank
- ~~MPI_Send~~ →
- ~~MPI_Recv~~ →
- MPI_Reduce
- MPI_Isend
- MPI_Irecv
- MPI_Wait
- MPI_Wtime
- MPI_Bcast

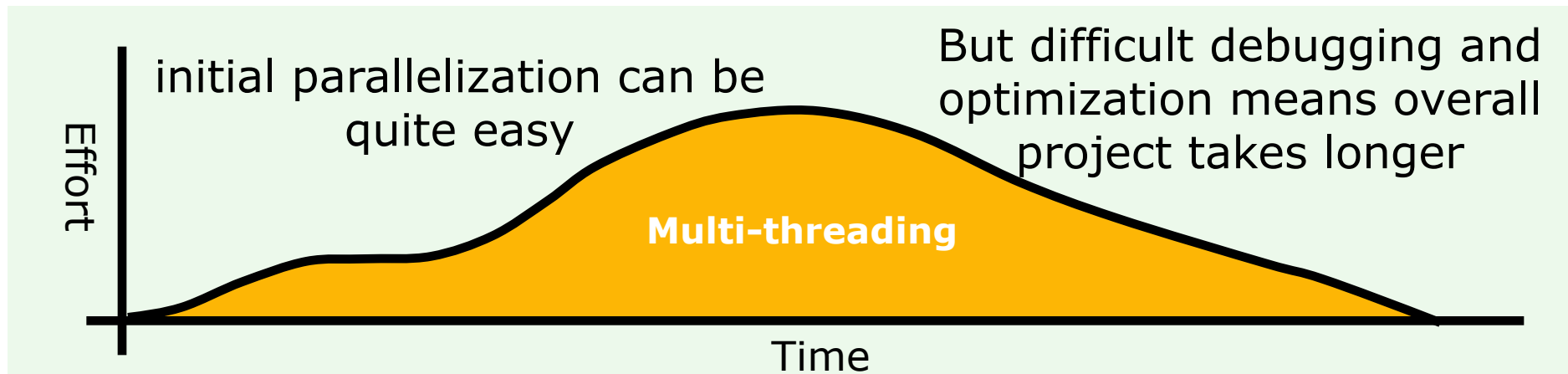
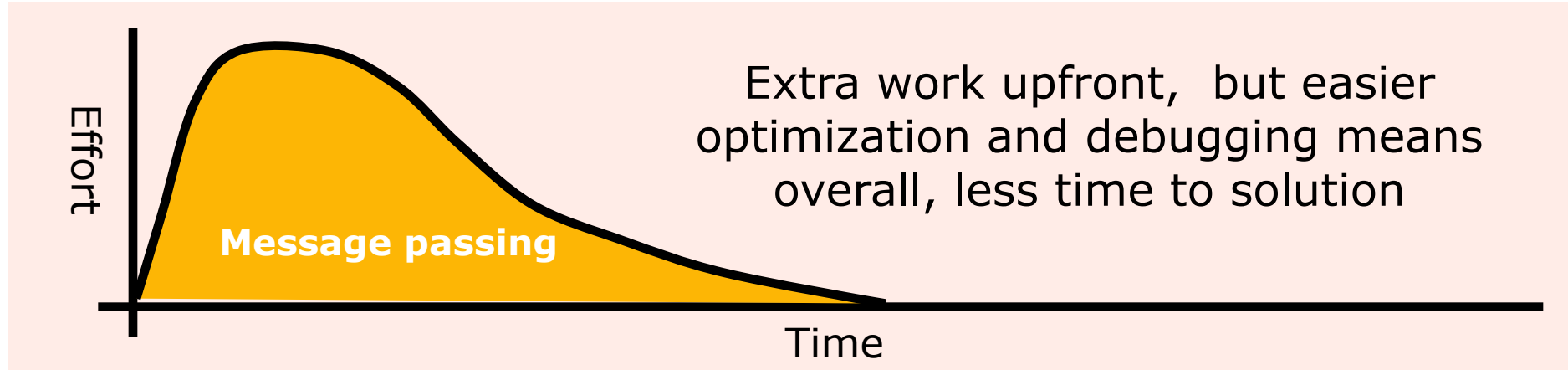
My friends on the MPI forum hate this slide.
These are indeed the functions most people use, but these date back to MPI 1.1 (Oct 1998) ... The spec is currently at version 5.0 (June 2025)

Real Programmers always try to overlap communication and computation .. Post your receives using MPI_Irecv() then where appropriate, use MPI_Isend().

Master these 12 constructs before exploring newer features in MPI.
Then learn about:

- Support for mixing MPI and OpenMP
- Topologies
- One-sided communication
- User defined types
- Shared memory programming within MPI (no need for OpenMP)

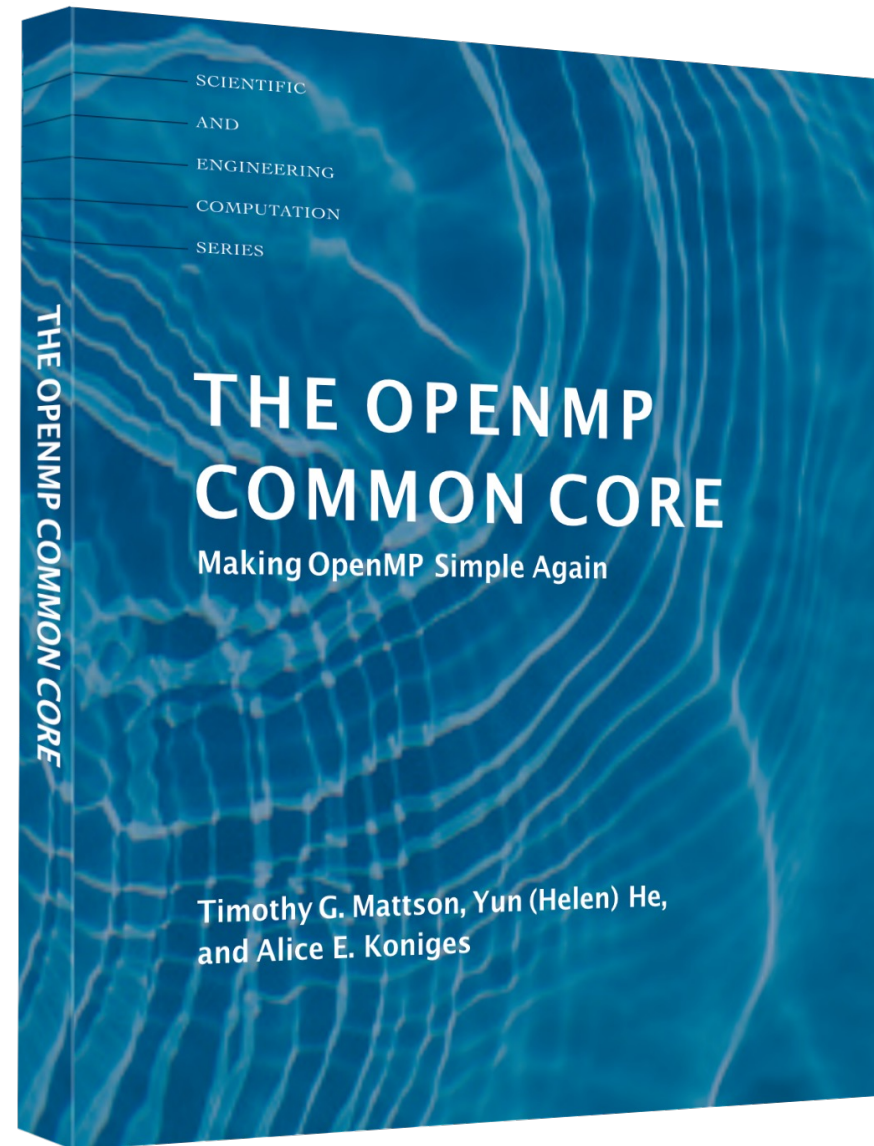
Does a shared address space make programming easier?



Proving that a shared address space program using semaphores is race free is an NP-complete problem*

To learn OpenMP:

- An exciting new book that Covers the Common Core of OpenMP plus a few key features beyond the common core that people frequently use
- It's geared towards people learning OpenMP, but as one commentator put it ... **everyone at any skill level should read the memory model chapters.**
- Available from MIT Press



Books about OpenMP

A great book that covers
OpenMP features beyond
OpenMP 2.5

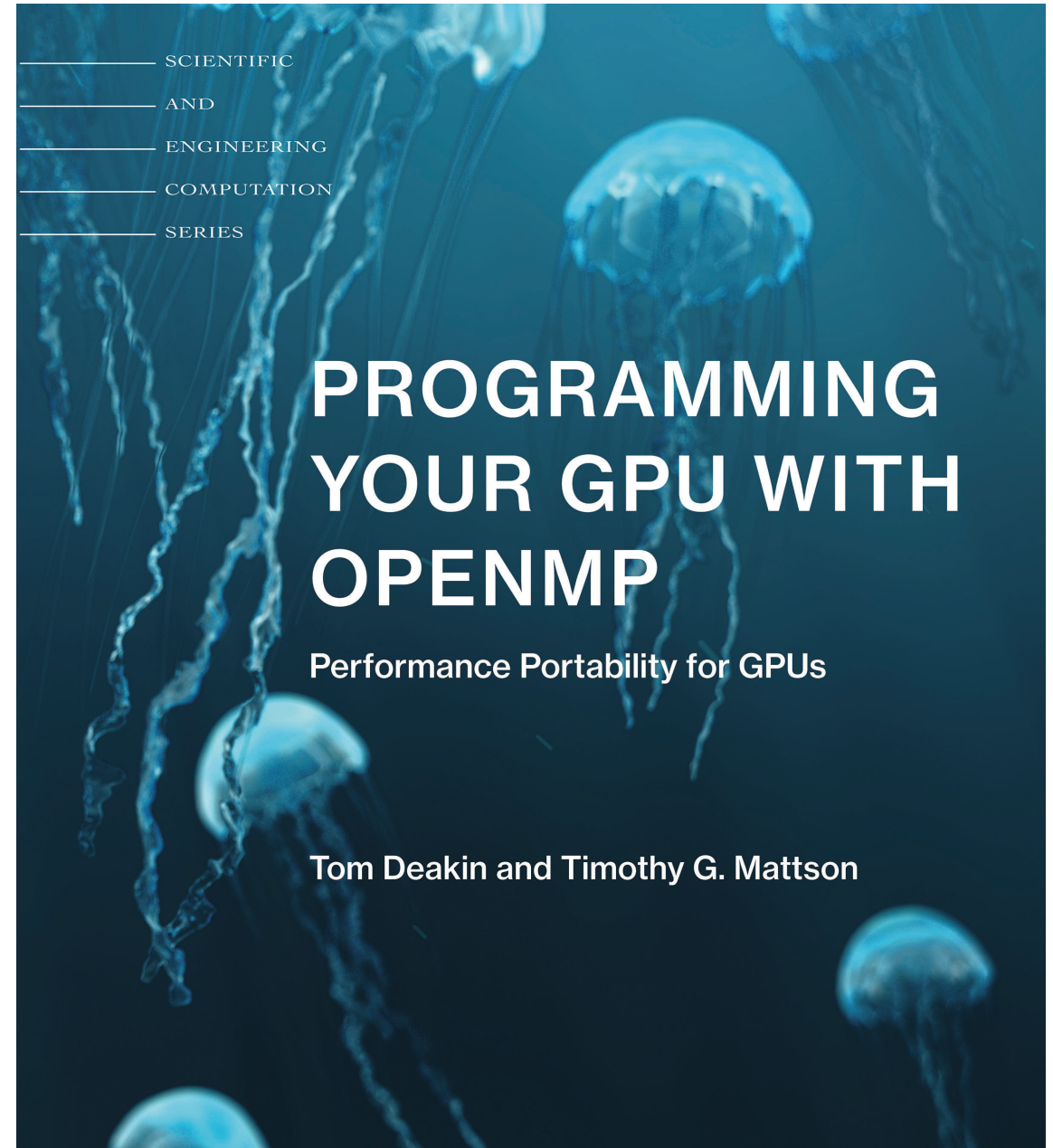


Books about OpenMP

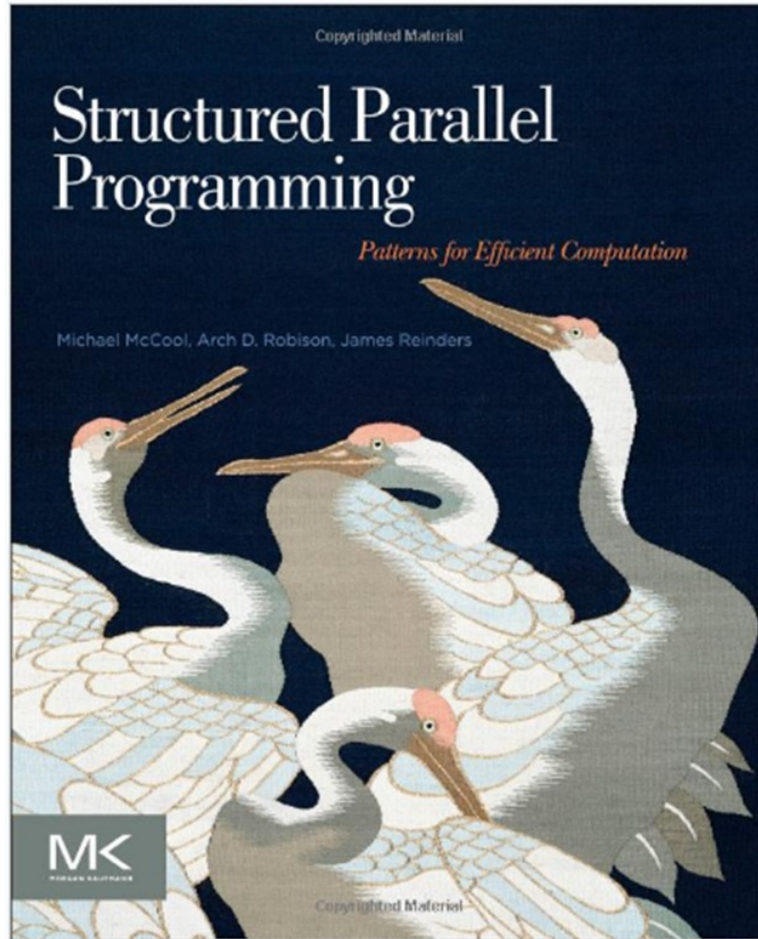
The latest book on OpenMP ...

Released in November 2023.

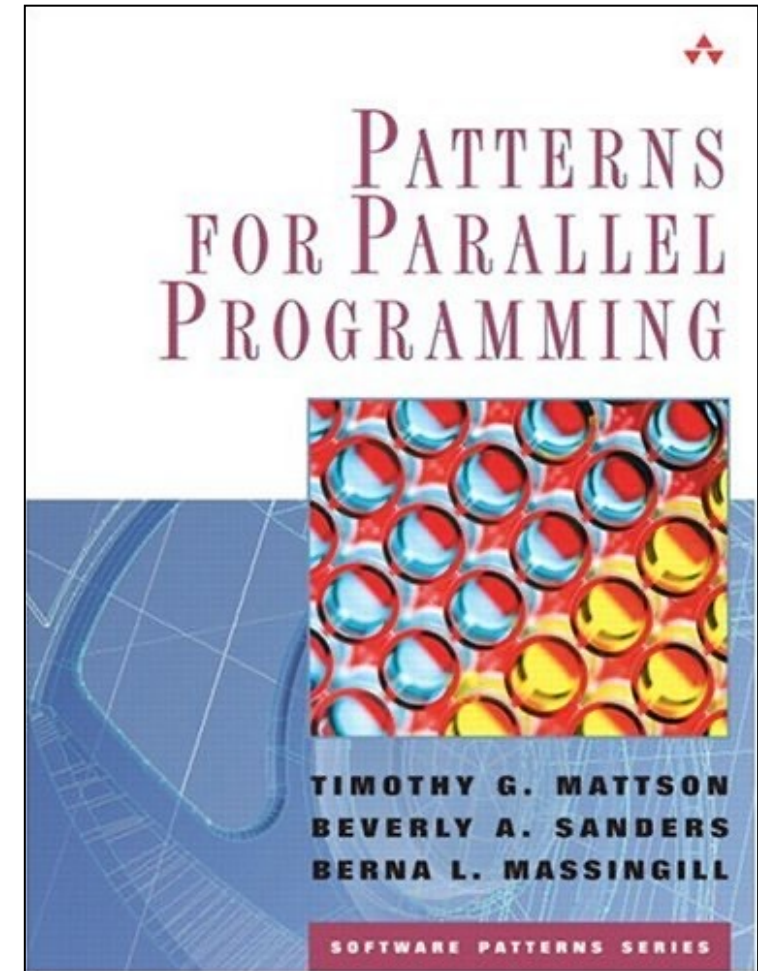
A book about how to use OpenMP to
program a GPU.



Background references



A great book that explores key patterns with Cilk, TBB, OpenCL, and OpenMP (by McCool, Robison, and Reinders)



- A book about how to “think parallel” with examples in OpenMP, MPI and java